TECHNICAL DOCUMENTATION · JUDGE PREPARATION

BIOS2026

The project, the engineering and the questions judges can ask

A local-first extension of the LocalFirst office suite

Evidence-labelled edition · 1 November 2026

Windows offline implementation · experimental Android relay · measured long-range and production gates still open

Based on source commit `d7e9135` and the available local verification reports.
Includes a technical handbook, sourced competitor comparison, minute-by-minute demo plan and 200 questions with model answers.

Contents

Use the PDF outline or these links to navigate. Source Markdown files remain the editable project documentation.

BIOS2026 / LocalFirst: Complete Technical Handbook

1. Project in one paragraph
2. What belongs to this project, and what came from upstream
3. Four modes that must not be mixed together
4. Technology stack and why each component exists
5. System architecture
6. Local data models and storage boundaries
7. Document creation and ordinary save logic
8. Opening, hydration and recovery logic
9. How shared editing uses Yjs
10. Synchronization messages and acknowledgement logic
11. The native Windows Bluetooth layer
12. Endpoint identity, invitations and cryptography
13. Browser-to-native bridge and API surface
14. Wire format, timeouts and size limits
15. Experimental Android relay: exact behavior
16. Fixed-route operation and its limitations
17. Backup, restore, installation and updates
18. Hosted features: how to describe them accurately
19. Evidence, test meaning and unresolved gates
20. Repository map and build workflow
21. Failure-oriented walkthrough
22. Terms presenters should be able to explain
23. Practical next engineering milestones
24. Source and evidence ledger

BIOS2026 / LocalFirst: USP and competitive landscape

1. The problem we are choosing to solve
2. The claim ladder: working, experimental and proposed
3. A USP that can withstand follow-up questions
4. What is inherited, and what this version contributes
5. Capability comparison
6. What to say about each alternative
7. Where a defensible advantage could develop
8. Target users and use cases: hypotheses to validate
9. Practical tradeoffs judges should hear
10. Business and deployment model without invented numbers

11. Metrics that would make the next pitch stronger
12. Hard judge challenges and defensible responses
13. A practical positioning checklist
14. Primary-source reference ledger

BIOS2026: presentation, demonstration and technical defence

1. The message to remember
2. Three presentation lengths
3. Ten-minute demonstration, minute by minute
4. Preparation checklist
5. Failure handling in front of judges
6. How to answer a difficult question
7. Operation and reproduction commands
8. Evidence to carry into the room
9. Next milestones and measurable acceptance
10. Presentation conduct and ownership

BIOS2026 / LocalFirst: 200 Judge Questions and Model Answers

1. Beginner questions: problem, purpose and scope
2. User experience and demonstration
3. Technology stack and engineering choices
4. Frontend architecture and mode boundaries
5. Local persistence, recovery and database behavior
6. CRDTs, journals and synchronization logic
7. Windows Bluetooth and local bridge
8. Encryption, authentication and trust boundaries
9. Android relays and cross-platform implementation
10. Failures, adversarial conditions and edge cases
11. Testing and evidence quality
12. Performance, scale and range
13. Build, deployment and operations
14. USP, alternatives and differentiation
15. Ownership, business and project responsibility
16. Expert traps and difficult follow-ups

Evidence and source map

PART 1 · TECHNICAL HANDBOOK

BIOS2026 / LocalFirst: Complete Technical Handbook

Edition: 1 November 2026. **Source baseline:** `d7e9135`. **Audience:** presenters, judges, maintainers and technical reviewers.

This handbook describes the source and available verification evidence. It separates the inherited office platform, the working Windows offline build, the implemented experimental relay software and the longer-range roadmap. A feature present in source is not automatically a feature validated on every device.

1. Project in one paragraph

BIOS2026 develops a local-first document workspace on top of the original open-source Diwan office suite. The current Windows package opens a browser interface from a local executable, stores documents on that device, and supports an explicitly shared Docs document over native Bluetooth. Local snapshots, recovery drafts and durable Yjs journals address different failure cases. A separate experimental Android app can forward a fixed route between selected Bluetooth neighbors while the document endpoints retain their original authenticated TLS session. Automatic mesh routing and an 800 m–1 km operating range are future goals, not completed measurements.

The useful engineering question is: **Can people keep creating and recovering their documents when internet service disappears, then exchange edits over an explicitly authorized local transport without giving forwarding devices the document keys?**

2. What belongs to this project, and what came from upstream

The repository is derived from vul-os/diwan, under the existing MIT OR Apache-2.0 license. The local inherited baseline was commit `8cdb6f5`. The contribution comparison to `d7e9135` shows 102 changed files, 7,717 insertions and 238 deletions, including the README merge. Those numbers describe the diff; they do not measure intellectual ownership or product quality.

Layer	Inherited foundation	Work added or changed in this version
Office UI	React application; Docs, Sheets, Slides, Whiteboards, PDF/signing surfaces	Offline status, recovery behavior, lazy loading, accessible controls and nearby-sharing UI
Editing	TipTap/ProseMirror, Yjs integration and other existing editor engines	Opt-in durable local replica lifecycle and native-nearby synchronization wiring
Hosted backend	Go/Gin, file APIs, storage adapters, authentication, ACLs, signing, rendezvous	Cloud/offline build and deployment support, plus a separate offline executable
Persistence	Hosted snapshots, revision checks, optional update logs; existing browser draft support	Transactional offline workspace, stronger draft acknowledgement, backup/restore and local Yjs journal
Nearby transport	Existing online WebRTC collaboration is a separate inherited path	Native Windows Classic Bluetooth RFCOMM, local bridge, pinned TLS invitation and Bluetooth setup guidance

Layer	Inherited foundation	Work added or changed in this version
Relay experiment	Standard Android and Go libraries	Fixed-route Android forwarder, distinct DRL1 admission, experimental receiver and bounded forwarding tests

A defensible attribution statement is: “We extended an existing open-source office platform to build and test this offline persistence and Bluetooth workflow.” Avoid “we wrote the whole office suite from scratch” or “we invented CRDTs, TLS or Bluetooth mesh.”

3. Four modes that must not be mixed together

Mode	Document persistence	Inter-device transport	Current interpretation
Hosted/self-hosted platform	Backend storage through HTTP APIs; optional server update log	Existing WebRTC peer collaboration with discovery infrastructure	Inherited platform capabilities; deployment-specific configuration matters
Windows offline workspace	Browser IndexedDB on `http://127.0.0.1:8765`	No transport needed for local editing; native Bluetooth for opted-in Docs	Current working package; Docs is the focus of the stronger verification
Android relay experiment	Documents remain on Windows endpoints; relay does not store document payloads	Fixed chain of paired Classic RFCOMM links containing the original endpoint TLS stream	APK and connector pass software checks; physical multi-link behavior remains to be validated
Future extended network	Intended durable endpoint replicas	Planned authenticated route management across participating relays	Automatic discovery/rerouting and 800 m–1 km range are unimplemented or unmeasured

`OFFLINE_ONLY` is a **build-time** flag: `VITE_OFFLINE_ONLY === 'true'`. It is not a login bypass that an ordinary hosted client enables at runtime. In that build, the API abstraction routes supported operations into a local adapter before selecting any cloud endpoint. Unsupported operations fail explicitly with status-like errors, usually 501, instead of silently making a network request.

4. Technology stack and why each component exists

4.1 Frontend and office engines

The table distinguishes the installed lockfile version from dependency ranges. A declaration such as `^2.11.7` is a permitted range, not proof that exactly version 2.11.7 is installed.

Technology	Lockfile version	Responsibility
React / React DOM	18.3.1	Component rendering, state-driven interface and lifecycle
React Router DOM	7.18.1	Routes such as `/docs/:id` and editor navigation
TypeScript	6.0.3	Compile-time checking of models, bridge messages and UI code
Vite	8.1.4	Module bundling, code splitting and production assets
Zustand	5.0.12	File list and native-nearby status stores

Technology	Lockfile version	Responsibility
TipTap / bundled ProseMirror	2.27.2	Structured rich-text editing, schema and transactions
Yjs	13.6.31	Mergeable shared document state and binary update encoding
y-prosemirror	1.3.7	Binding between ProseMirror structure and the Yjs fragment
FortuneSheet React	1.0.4	Existing spreadsheet interface
Excalidraw	0.18.1	Existing whiteboard interface
Tailwind CSS	Dependency range `^3.4.17`	Utility styling and responsive layouts
DOMPurify	Dependency range `^3.4.5`	Existing HTML sanitization boundaries

Import/export libraries include SheetJS `xlsx` 0.20.3, Mammoth 1.12.0, `docx` 9.6.1, `pdf-lib` 1.17.1, PDF.js 4.10.38 and PptxGenJS 3.12.0. Their presence enables particular format workflows; it does not prove perfect Microsoft Office round-trip fidelity for every document.

4.2 Native, backend and storage stack

Technology	Version or choice	Use
Go	`go 1.25.0`; requested toolchain `go1.25.12`	Native helper, backend and typed concurrency
Gin	1.9.1	Existing hosted HTTP server; the small offline server uses Go `net/http` directly
Go `crypto/tls`, `crypto/x509`, `crypto/rand`	Standard library from the selected Go toolchain	Endpoint TLS, ephemeral certificate and unpredictable secrets
Windows Winsock / AF_BTH / RFCOMM	Native Windows APIs	Actual Classic Bluetooth sockets, SDP registration and paired-device access
IndexedDB	Browser API	Offline snapshots, draft records and replica journals
PostgreSQL driver `pgx/v5`	5.10.0	Existing hosted PostgreSQL storage option
`modernc.org/sqlite`	1.50.1	Existing hosted auxiliary stores; not the offline browser document database
Prometheus client / OpenTelemetry	1.23.2 / 1.43.0	Existing hosted observability support
Android application	Java 17, minimum API 26, target/compile API 36	Experimental native relay service and permission-aware UI
Android Gradle Plugin	8.13.0	Android compilation, packaging, lint and debug signing
Gradle	8.13	Build orchestration for the Android experiment

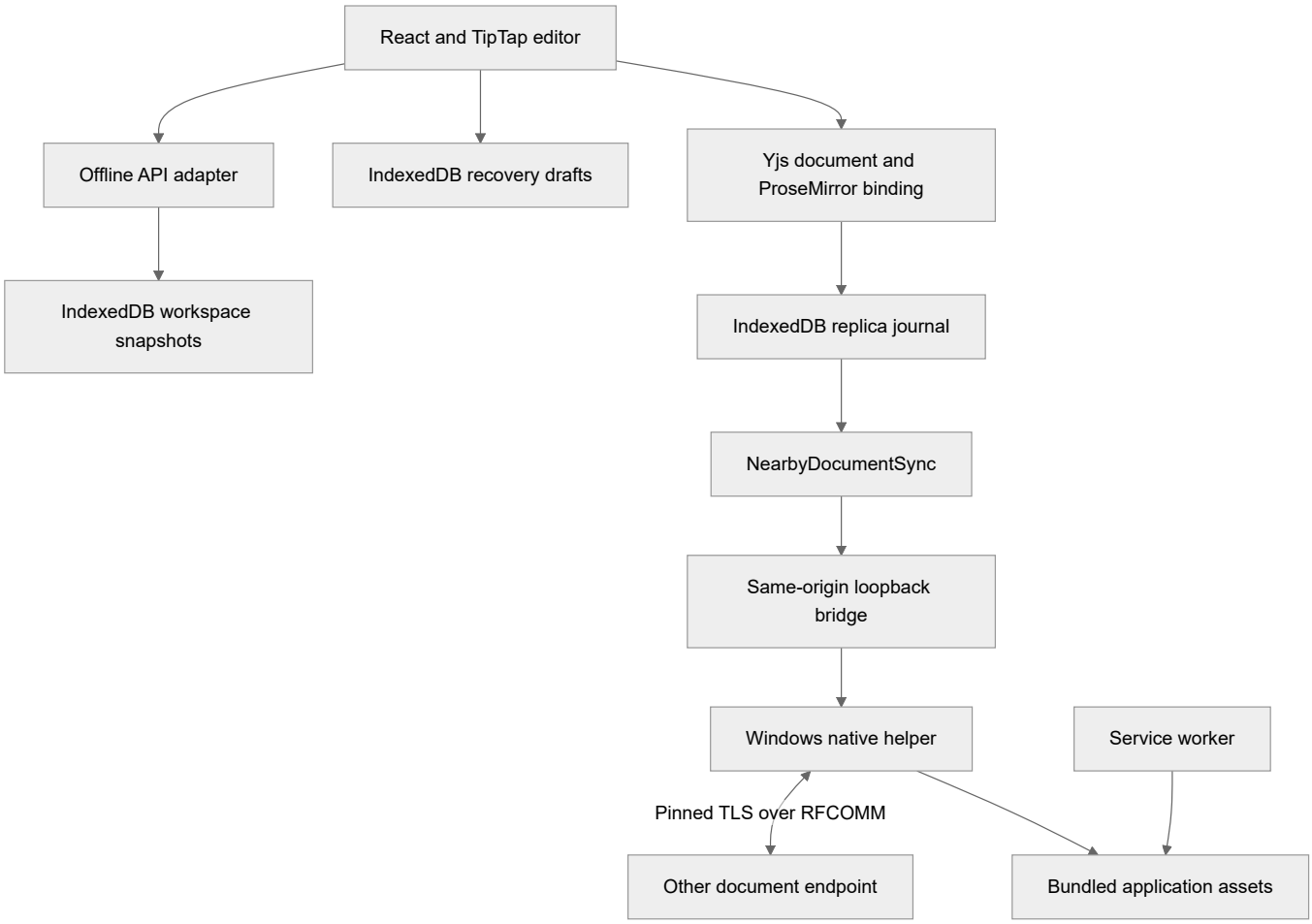
An important distinction: Node.js and the JDK are development tools. Users of the packaged Windows offline executable do not need Node.js or Go installed. Building the Android APK needs a local SDK/JDK/Gradle environment; running the installed APK does not require those development tools on the phone.

4.3 Test stack

Vitest 4.1.7 tests browser-side logic. Playwright Test 1.61.1 drives real Chrome scenarios. Testing Library tests React behavior, fake-indexeddb supplies a controlled IndexedDB implementation, and MSW supports request-level integration tests. Go tests inject memory or loopback transports to separate protocol behavior from radio hardware. JVM tests exercise the Android forwarding core without pretending to exercise the Android framework.

5. System architecture

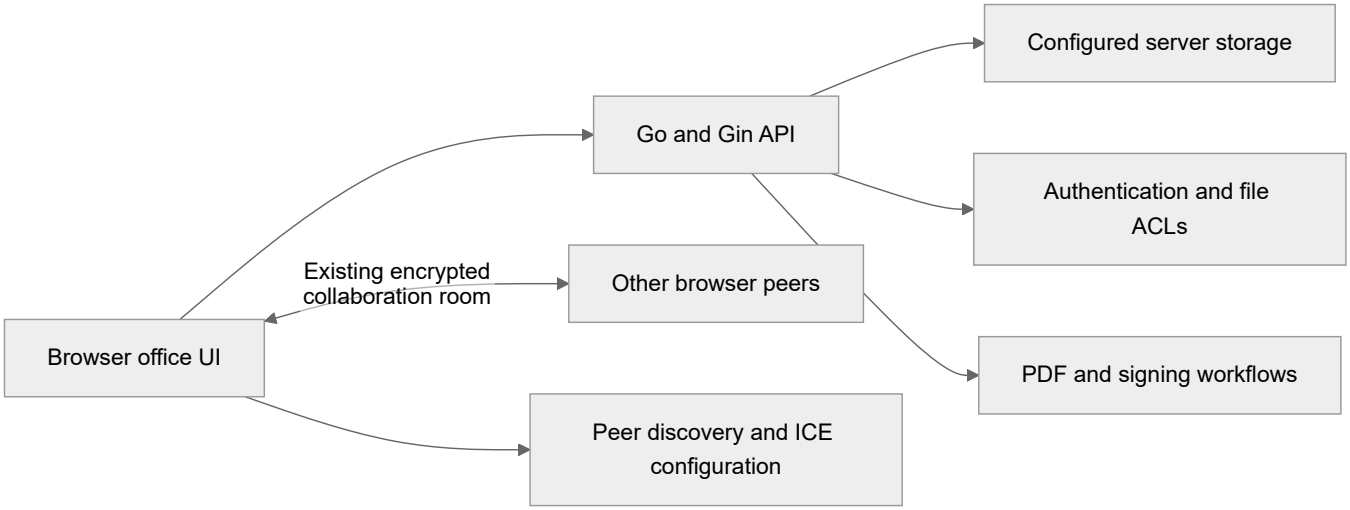
5.1 Windows offline architecture



The browser owns editing and local document persistence. The Go helper owns platform radio access, socket lifetimes and native authentication. The service worker owns application assets, not private document records. This separation permits local editing to continue even when the radio is unavailable.

`127.0.0.1` means “this computer.” Each laptop opens its own local server. The two local HTTP servers do not exchange document edits over a Wi-Fi LAN; native Bluetooth sockets carry the inter-laptop path in the offline preview.

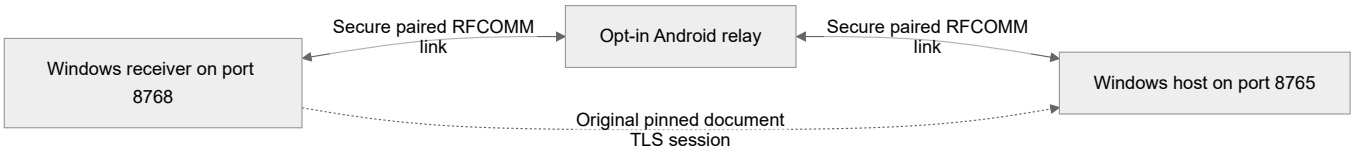
5.2 Hosted architecture



The hosted system can persist whole-document snapshots and metadata on the server. Its CRDT collaboration path is distinct from ordinary REST storage. End-to-end encryption of collaborative transport does **not** by itself establish that every server snapshot, export or database record is encrypted from the server administrator. Documentation must identify which data path a privacy claim refers to.

The main storage factory selects local or PostgreSQL storage, with `DATABASE_URL` or `VULOS_DATABASE_URL` able to select PostgreSQL. A separate S3-compatible configuration/client layer supports Tigris or MinIO-style object storage concerns; it must not be described as the active offline database. Authentication, ACLs, comments, suggestions, signing and observability belong to the hosted platform unless their offline paths are specifically implemented and verified.

5.3 Experimental relay architecture



The phone forwards an opaque reliable byte stream after separate relay admission. The original document TLS session terminates on the two document endpoints. The phone is not a document editor, TLS document endpoint or automatic internet gateway.

6. Local data models and storage boundaries

The offline workspace database is `diwan-local-workspace`, schema version 1. It has four stores: `files`, `folders`, `versions` and `metadata`. File IDs and folder IDs are explicit keys. The version store also indexes `file_id` so history can be retrieved without interpreting the document body.

Model	Important fields	Meaning
Local file	`id`, `name`, `type`, `content`, `rev`, `owner`, `parent_id`, `starred`, `trashed`, timestamps	Saved document snapshot and workspace state

Model	Important fields	Meaning
Nearby flags	`nearby_enabled`, `nearby_initial`	Opt-in replica mode and a newly accepted document awaiting its initial replica
Local folder	`id`, `name`, `parent_id`, `trashed`, `created_at`	Workspace hierarchy
Local version	`id`, `file_id`, `name`, `content`, `rev`, `created_at`, `label`	A bounded prior snapshot
Device metadata	`key`, `id`, `created_at`	Device-local identity metadata, not an invitation secret

The draft database is separately named `diwan-drafts`, with a `drafts` store. A draft contains the file ID, title, content, timestamp and an acknowledgement token. The replica database is `diwan-replicas`, with a `checkpoints` store keyed by file ID and an auto-incrementing `updates` store indexed by file ID.

These are separate persistence responsibilities:

1. **Snapshot:** the saved JSON representation the workspace can list and export.
2. **Draft:** a recovery copy around an interrupted ordinary save.
3. **Journal:** binary Yjs operations needed to recover and merge a shared replica.
4. **Application cache:** JavaScript, CSS, fonts and other assets needed to open the app.

Clearing the service-worker cache is not the same as clearing IndexedDB. Clearing all browser site data can remove both. Changing browser profile, host spelling or port changes the storage origin, which can make existing documents appear absent even if the old origin still has them. Port 8768 deliberately has a different experimental workspace from port 8765.

7. Document creation and ordinary save logic

Creation validates a supported file type, a non-empty trimmed name and the content-size bound. It assigns a random UUID, initial revision 1 and timestamps, then writes the record in a transaction. The browser displays a successful save only after the relevant transaction completes.

For a normal unshared document, saving is snapshot-based. The client sends its expected `rev`. The read/write transaction reads the current file and rejects a stale revision rather than silently overwriting another tab's committed work. A changed snapshot increments the revision and records the previous snapshot in history. An unchanged name/content pair does not create a meaningless new revision. History retention is bounded to 50 versions.

```
save ordinary document:
    capture current dirty generation
    mark save state as saving
    write recovery draft and receive its unique token, if storage permits
    read current saved record inside a write transaction
    reject if file is missing, is now replica-enabled, or revision is stale
    write next snapshot and bounded prior history
    wait for transaction completion
    clear only the draft whose token belongs to this save
    show saved only if no newer dirty generation appeared
```

The token solves a subtle race: save A may finish after the user has already produced draft B. Acknowledging A must not delete B. Dirty generations solve the related UI race: finishing an old save must not label later keystrokes as saved.

In the offline Docs editor, the ordinary content-update debounce is 250 ms. That is a scheduling delay, not a measured save-latency guarantee. Title, page setup and other changes have their own scheduling paths. Ctrl+S and Save trigger explicit attempts. Navigation handlers attempt preservation and warn while work is pending. A browser crash, sudden power loss or mobile process termination can still remove edits that never reached durable storage.

8. Opening, hydration and recovery logic

An editor must know which document and draft it is opening before it can become editable. Otherwise, a blank initial editor state can race with data loading and overwrite real content. This version gates editing and saving while hydration or recovery remains unresolved.

If a draft is found, the user receives a recovery decision. Restore preserves the recovered content; discard is an explicit user choice. Missing documents show a visible error rather than creating a blank replacement under the same ID. If storage or rendering fails, the UI keeps the failure visible and offers appropriate recovery actions. The app error boundary provides a path away from a blank screen, but an error boundary is not proof that every asynchronous failure is recoverable.

If an old tab attempts to save a snapshot after that file has been opted into shared-replica mode, the local storage layer rejects the old save. The old editor must recover its work and reopen the document instead of overwriting the journal-backed state with a stale snapshot.

9. How shared editing uses Yjs

A CRDT is a data structure designed so compatible replicas can merge operations without a central editing authority. Here, Yjs tracks structured document state. The actual rich-text document lives in a `Y.XmlFragment` named `prosemirror`; y-prosemirror translates between this structure and ProseMirror transactions.

Sending only `editor.getText()` would lose headings, marks, tables, links and other structure. Raw text offsets also do not map reliably to positions inside a structured document. Binary Yjs updates carry the shared structure rather than a plain-text diff. Existing metadata such as title, page setup and header/footer settings can live in the shared `diwan-meta` map.

CRDT convergence does not mean every conflict produces the result a human intended. Concurrent title edits, deletes and formatting changes have defined merge behavior, but the UI still needs understandable history and recovery. CRDTs also do not provide authentication, transport encryption, storage durability or malicious-input validation; those are separate layers.

9.1 Opt-in migration

Choosing Share this document first waits for document readiness and successful saving. It checks the 512 KiB source-JSON sharing limit. Only then is the file marked `nearby_enabled`, and the editor reloads into its persistent replica lifecycle. Other ordinary documents continue using their normal snapshot behavior.

When opening a replica, the code reads a checkpoint and journal records, applies them to the Yjs document and checks validity. If no journal exists, it seeds a replica from the saved document and metadata. Initialization merges records inside the write transaction so two tabs opening the same file cannot each overwrite the other's initialization.

9.2 Journal commits and compaction

The local replica subscribes to Yjs updates. It copies each update and serializes persistence through a promise tail. Updates larger than the replica limit, or a resulting encoded state larger than that limit, fail visibly. A successfully committed update can then be announced to synchronization listeners.

After roughly 128 appended updates in that replica instance, compaction merges the current state with the committed checkpoint and all committed update records for that file. The checkpoint replacement and covered-record pruning happen in the same IndexedDB write transaction. Including other tabs' records matters: checkpointing only one tab's current memory could otherwise prune operations that tab has not received yet.

`flush()` waits until the persistence tail stops changing. Awaiting a single old promise is insufficient if a new edit appends another write while that promise is pending. Local tabs use BroadcastChannel for already committed updates; consumers replay them without treating every local broadcast as a new journal append.

10. Synchronization messages and acknowledgement logic

The nearby document protocol has three message kinds:

Kind	Core fields	Purpose
`vector`	`fileId`, `data`	Advertise Yjs state knowledge to request missing changes
`update`	`fileId`, `id`, `data`	Transfer an encoded update with a unique acknowledgement ID
`ack`	`fileId`, `id`, `data`	Acknowledge the referenced update after destination persistence

Binary values are base64 encoded in the browser/native JSON envelope. Base64 is an encoding; it is not encryption. Encryption is provided by the authenticated endpoint channel beneath this envelope.

On connection or a detected event reset, the synchronization layer starts a new generation and exchanges state vectors. The receiver computes a delta with `Y.encodeStateAsUpdate(document, peerVector)`. Updates are validated against the expected file and size bounds, shadow-applied and checked against the real document schema before being applied to the live document. The receiving replica then flushes its journal before sending an acknowledgement.

```
receive update:
    reject wrong file, malformed ID, invalid base64 or excessive size
    validate the prospective document through the schema boundary
    apply the valid update to the local Yjs document
    await the destination replica's fully drained durable journal
    ensure connection generation is still current
    send acknowledgement for this update ID
```

The UI distinguishes local, syncing, acknowledged and error states. “Saved on this device” means local persistence. “Other laptop acknowledged this version” is a stronger, destination-specific statement. A connected Bluetooth socket, a relay byte count or a successful send call proves neither equivalent document contents nor destination storage commit.

10.1 Why a state vector alone is insufficient

Deletion information requires care. The synchronization code explicitly avoids treating matching state vectors alone as proof that all deletions have been durably received. Outstanding updates have sequence numbers and a `coversEarlier` marker. A full-state durability barrier can retire older outstanding acknowledgements only after

its own acknowledgement. When the outstanding map reaches its bound of 128, tracking is replaced with such a full-state barrier rather than forgetting unacknowledged changes. The acknowledgement path also checks the current generation, current pending work and current state-vector value before reporting an acknowledged state.

The sync layer batches persisted outgoing updates with a short 25 ms timer and periodically requests resynchronization every eight seconds. These constants bound or schedule work; they are not guarantees of eight-second recovery or 25 ms end-to-end latency on physical radios.

11. The native Windows Bluetooth layer

Browsers do not provide this application's general Classic Bluetooth socket and service-listener functions. The native helper supplies them using Windows APIs. The implementation uses AF_BTH sockets, `SOCK_STREAM`, RFCOMM protocol selection, service registration and native device enumeration. It requests authentication/encryption-related socket options, restricts outgoing selection to OS-authenticated paired devices and does not silently pair arbitrary devices.

The document service UUID is `d71a5670-e6ec-4a22-87f1-b0ed9b5dc321`. SDP lets the peer find the corresponding service/channel; the channel is not an internet TCP port. The Windows packed Bluetooth address structure is 30 bytes. Dedicated native tests check wrappers and behaviors that would otherwise be easy to get wrong, including partial reads/writes, closure and deadlines.

The hub serializes commands, guards shared state with a mutex and serializes packet writes. Its generation counter prevents a late handshake from a stopped session publishing a new “connected” state. Both pending authentication sockets and established sockets are owned so Stop can interrupt them. Hosting handles one peer for one document at a time; it is not a many-user document-room implementation.

The state machine is `idle → listening/connecting → connected`, with errors, disconnections and expiry returning visible failure or listening states as appropriate. Client reconnect attempts are bounded by a session context; retry scheduling does not recreate an expired invitation automatically. Real simultaneous radio and lifecycle behavior still needs device validation.

12. Endpoint identity, invitations and cryptography

12.1 DBT1 invitation construction

For a hosting session, Go generates an ECDSA P-256 key and self-signed certificate. It hashes the exact certificate DER bytes with SHA-256 and generates a separate random 32-byte invitation secret. The invitation is:

```
DBT1- + base64url_without_padding(certificate_SHA256_pin[32] || secret[32])
```

The resulting code has a five-character prefix and an 86-character encoded body. Its JSON file is tagged `localfirst-bluetooth-invitation`, version 1. A workspace backup is a different format and must not be accepted as an invitation.

The receiver creates a TLS configuration with a minimum of TLS 1.3. Ordinary public-CA verification is replaced by a `VerifyConnection` callback that requires exactly one presented certificate and constant-time equality with the pinned hash. The source sets `InsecureSkipVerify` for that custom trust model; the security property depends on the callback being present and correct. It must never be copied into code as “skip verification and accept anything.”

The TLS client then sends the invitation secret inside the established encrypted channel. The host checks it in constant time before sending the document offer. Thus, the pin authenticates the intended host identity, while possession of the secret authorizes joining that session. The invitation is a bearer capability, not a named user account or a public certificate-directory identity.

12.2 What is and is not a key

The invitation secret and certificate pin are not the TLS traffic-encryption key. TLS negotiates traffic secrets between the actual endpoints. Do not tell a judge that the JSON invitation contains the raw AES document key. Do not assert one particular negotiated TLS cipher suite unless a specific run records it; Go's TLS implementation negotiates supported TLS 1.3 suites.

The certificate validity is approximately 24 hours, while the hub session/invitation lifetime is 30 minutes. Those are different controls. Expiry ends native session use; it does not revoke documents already copied to a recipient, retract exported files or securely erase endpoint memory. A fresh share generates a new certificate/secret invitation.

12.3 Threat boundaries

The design aims to protect document contents against passive interception and unauthorized peers on the transport. It does not establish encryption at rest for browser databases or JSON backups. Someone controlling an unlocked endpoint profile can read local documents. A stolen invitation plus usable paired access can permit joining while the session remains valid. A compromised endpoint, recipient copying plaintext, radio jamming and deliberate packet dropping cannot be solved by transport encryption alone.

Relays can observe adjacent devices, traffic volume, timing and disconnections. Content confidentiality is not anonymity or zero metadata leakage. There is no completed independent security audit, formal security proof, post-quantum certification or production-security certification in the available evidence.

13. Browser-to-native bridge and API surface

The browser contacts only its own origin through `/_nearby/`. Bootstrap obtains an unpredictable local helper token. Subsequent requests include `X-LocalFirst-Native-Token`; fetch is same-origin, omits browser credentials, refuses redirects and avoids caches.

Route	Method	Purpose
`/_nearby/bootstrap`	GET	Return bridge token, transport identifier and protocol version
`/_nearby/status`	GET	Return actual hub status and radio state
`/_nearby/devices`	GET	Return paired Bluetooth devices
`/_nearby/events?after=...`	GET	Long-poll events and status after a cursor
`/_nearby/host`	POST	Begin hosting the chosen saved document
`/_nearby/connect`	POST	Connect to selected device using the invitation
`/_nearby/send`	POST	Send a validated message for the active document
`/_nearby/stop`	POST	Stop pending and active native session work

The handler rejects non-loopback Host values, mismatched Origin and inappropriate Fetch-Site headers. Except bootstrap, operations require the token. POST bodies must be JSON, fit the size ceiling, contain known fields and contain exactly one JSON command. Authentication and validation are independent of the editor UI; disabling a button is not the security boundary.

The event queue retains at most 128 events or about 8 MiB of message bytes. Long polls wait for up to 18 seconds. If a client has fallen behind retained events, a reset flag causes resynchronization rather than assuming every missed update arrived. The queue is transport observation, not the durable document journal. Transport session IDs filter old events.

The offline server's other `/api` and `/v1` document requests are rejected. Its special `/api/offline-launch`, script and health routes are startup exceptions; they are not cloud document APIs. The hosted server has a much wider API surface, including files, folders, versions, authentication and signing. A route-shaped call handled by the local adapter is not necessarily an HTTP request.

14. Wire format, timeouts and size limits

On the authenticated stream, native packets are JSON preceded by a four-byte unsigned big-endian length. The reader rejects zero and oversized lengths before allocating the packet buffer; it uses `io.ReadFull` to handle fragmented stream delivery. The writer loops until the complete length-prefixed buffer is written or fails. RFCOMM is a stream: one socket read is not guaranteed to equal one application message.

Bound	Current value	Layer and interpretation
Ordinary local document content	16 MiB	Encoded JSON content validation
Source document before Bluetooth opt-in	512 KiB	Serialized source JSON; stricter than the ordinary local limit
Shared replica / checkpoint / nearby decoded update	1 MiB	Current native-nearby replica boundary
General existing CRDT envelope default	128 KiB	Separate inherited envelope bound; do not replace the nearby 1 MiB figure with it
Native packet JSON	2 MiB	Length-prefixed native framing and bridge command bound
Browser invitation input/file	4,096 bytes	Small-file validation; rejects backups and reports
Native host document title	2,048 bytes	Native document offer validation
Workspace name	512 characters	Local file/folder naming
Native document data field	1,500,000 encoded characters	Additional native message validation
State vector	65,536 decoded bytes	Nearby synchronization validation
Native event retention	128 events / 8 MiB	Bounded, non-durable bridge queue
Outstanding update tracking	128	Replaced only through a full-state barrier
Host/join session	30 minutes	Session-context lifetime

Bound	Current value	Layer and interpretation
Native handshake/operation deadline	12 seconds in relevant paths	Failure bound, not performance promise
Native heartbeat/read timeout	8 seconds / 30 seconds	Connected-session liveness behavior
Backup restore input	32 MiB	Entire backup JSON input
Backup record ceilings	1,000 files; 1,000 folders; 50,000 versions	Parser limits, not recommended workload sizes
Backup content complexity	Depth 64; 200,000 visited values	Bounds malicious or corrupt nested content
Retained local history	50 versions per file	Snapshot history retention

Limits constrain supported work and resource exposure. They do not prove a malicious peer cannot consume excessive CPU within those bounds. Large-document memory, throughput, timing and denial-of-service testing remain release work.

15. Experimental Android relay: exact behavior

The app's package ID is `org.diwan.relay.experimental`. Its UI lists already paired devices; it does not scan nearby phones, request location permission or silently pair. The owner chooses an incoming neighbor and an outgoing neighbor, enables explicit consent and starts relay mode. The service creates an ongoing foreground notification with Stop and returns `START_NOT_STICKY`; it is not configured to restart secretly after process death.

The relay service UUID is `5bb53883-7094-461b-9d28-f7189bb4cb62`, separate from the document service. A fresh 32-byte relay token is displayed as `DRL1-` plus a canonical unpadded base64url body. It is distinct from DBT1 and cannot authenticate to the document host.

Admission uses exactly the four magic bytes `DRL1` followed by the 32-byte relay token. The phone checks the selected incoming Bluetooth address and bonded state, then checks this token. It creates an outgoing secure RFCOMM socket only for its configured next device. If the next device is another relay, it performs that relay's separate admission first. A one-byte ready indication is returned upstream only after the outgoing link or next relay is ready.

After admission, two copy workers forward bytes in opposite directions. A shared atomic budget prevents both directions collectively exceeding the configured allowance. Buffers are cleared on exit, the app does not log payload bytes or persist them, and counters track forwarding rather than destination commit. Java byte arrays can be cleared, but managed-runtime copies and immutable UI strings prevent any claim of perfect memory erasure.

Current constraints are one circuit per phone, 16 KiB copy buffers per direction, 64 MiB per-circuit shared byte budget, 30-minute service lifetime, 12-second setup and 30-second inactivity bounds checked by a fixed-delay watchdog. The code rate-limits admitted setup attempts over a window; this is not a complete hostile-radio denial-of-service defense. Socket ownership and cleanup handle permission/radio exceptions. Cleanup does not substitute for physical lifecycle tests.

The manifest requests legacy Bluetooth permissions only up to API 30, modern `BLUETOOTH_CONNECT`, notification access, foreground-service access and the connected-device foreground-service type. The verified APK has no Internet permission. The app disables backup/extraction for its data, marks the service non-exported and

prevents ordinary screenshots of its sensitive setup screen. None of these controls establishes device-compromise resistance.

No wake lock or battery-optimization exemption is requested. Screen-off behavior, Doze, OEM process management, revoked permissions and force-stop therefore require physical tests. A visible notification expresses foreground operation; it is not a guarantee that Android will keep every radio path alive indefinitely.

16. Fixed-route operation and its limitations

For the first trial, the topology is receiver laptop → phone → hosting laptop. The host uses the stable package and keeps its document open. The original DBT1 document invitation travels directly to the receiver. The phone chooses those two laptops as its neighbors and gives only its DRL1 relay code to the receiver's separate experimental launcher. That receiver uses port 8768, selects the phone in the Bluetooth dialog and supplies the host's original DBT1 invitation.

For two phones, configure backward from the host. Phone B forwards toward the host; phone A forwards toward B using B's relay code; the receiver contacts A using A's relay code. The routing path is chosen manually. The code does not discover an optimal path, bypass missing relays, recruit strangers' devices, perform opportunistic store-and-forward or provide a Bluetooth SIG Mesh implementation.

A disconnected path can leave both endpoints with local edits. Recovery requires fresh endpoint authentication as needed and missing-update exchange from durable endpoint journals. Restarting a relay generates a new DRL1 token, so the incoming device's configuration must be updated. The experiment does not silently fall back to cloud or Wi-Fi when a path breaks.

An 800 m route requires a working link between every adjacent pair. Node count depends on measured spacing, obstacles, radios and redundancy. A straight route to one point is not an 800 m radius of coverage. For illustration only, 800 m divided by a **measured reliable** 40 m spacing gives 20 links and 19 intermediate nodes before adding margin. That arithmetic is not a claim that these phones provide 40 m links.

17. Backup, restore, installation and updates

Export workspace produces a `diwan-local-workspace` version-1 JSON snapshot of saved files, folders, history and metadata. It does not export the service-worker asset cache or directly serialize the separate replica-journal database. Save current work first, especially after recent shared edits. An exported file is not a Bluetooth invitation, an encrypted-at-rest vault or a promise that the latest uncommitted keystroke is included.

Restore validates the entire input before writing. It checks schema/version, record counts, names, identifiers, revisions, timestamps, duplicate keys, missing parents, folder cycles, history references and bounded content complexity. It remaps file/folder IDs and adds new copies in a transaction instead of replacing current files. Device identity is retained. Restored snapshots do not silently inherit a live collaboration session.

The offline build bundles the application assets, applies a restrictive Content Security Policy and generates a content-derived service-worker version. Runtime `connect-src 'self'` prevents external connection targets from the offline page; the bridge remains same-origin. This does not mean the development toolchain needs no internet to download dependencies initially.

The Windows package includes a native executable, `web` assets, launchers and license notices. It must be fully extracted; running a launcher directly inside a ZIP is not a supported installation. The helper binds to loopback only and rejects public listen addresses. Documents remain in the browser profile, not in the extracted package directory.

An earlier upgrade failure left the browser showing an old cached interface although the native server had been replaced. The startup page now lives under `/api/offline-launch`, which earlier released workers bypass. It registers/updates the worker, requests activation, verifies the expected version and navigates to Home. Existing editor tabs are deliberately not forcibly reloaded. Older hashed assets can remain readable while old tabs need them. The build digest includes worker-generation code so lifecycle fixes also change the advertised build version.

18. Hosted features: how to describe them accurately

The broader platform includes Docs, Sheets, Slides, Whiteboards and PDF/signing. Hosted file APIs support CRUD, folders, revisions and related workflows. Existing authentication and per-file authorization protect hosted operations when configured. The codebase includes PostgreSQL, local-storage and auxiliary SQLite concerns, plus object-store configuration, metrics and tracing integrations.

Existing online collaboration uses its own peer room and WebRTC transport. Discovery/signaling, ICE configuration, rendezvous and possible content-blind fallback are online-network concerns. They are not used to carry the offline Bluetooth demo's document edits. Signaling can expose connection metadata even when document payloads are encrypted. A hosted persistence API may receive plaintext snapshots; do not combine transport and storage claims into “no server can ever see anything.”

The offline build disables the ordinary online Docs collaboration and server update-log flags. Its native-nearby lifecycle is a separate path. Several comments and older documentation predate that implementation and still describe nearby transport as pending. Current source and dated test evidence take precedence over stale prose. In particular, do not present every inherited menu or library as an offline-tested feature. Docs has the concentrated offline evidence; other editors, complex imports, attachments, signing and multi-user cloud workflows need their own verification matrices.

19. Evidence, test meaning and unresolved gates

Evidence	Available result	What it does not establish
Packaged Windows report	19 Chrome scenarios; no failed, skipped or flaky cases in that recorded run	Every editor, all browsers, all Windows adapters or power-loss recovery
Focused browser-logic audit	Historical record of 133 focused tests across 13 files	A current full-suite count or 133 physical scenarios
Single-laptop hardware probe	Radio availability, paired-device enumeration, listener/SDP registration/removal	A document sent between two physical laptops
Packaged owner test	Real owner start/stop and native-generated invitation export/import	Verified remote peer delivery on another radio
Upgrade regression	Reproduces old cached Home; activates new shell while preserving old editor/saved document	Every future migration or every browser profile
Phase-one relay lab	23 leaf scenarios × 10 runs = 230 executions, up to 32 simulated relays	230 distinct cases, 32 physical devices or any measured distance
Android core JVM tests	108 checks	Android permissions, service lifecycle, OEM background behavior or actual radio forwarding

Evidence	Available result	What it does not establish
Tagged Go experiment tests	9 leaf scenarios × 3 runs = 27 executions	A physical Android relay route
APK checks	26,367-byte debug-signed APK; signature verified; lint has zero issues; expected permissions only	Production signing, store acceptance, device support or a security audit
Experimental Windows smoke	Isolated launcher/banner/Bluetooth dialog; no external/document API requests	Remote Bluetooth connection or endpoint convergence on physical devices

The recorded race-detector run is **not passed**: the installed MinGW compiler cannot build the Windows amd64 race runtime. Ordinary tests and `go vet` do not replace that gate. Independent security review, production signing, supported-device matrices, clean disconnected installation, long-duration load, p95/p99 latency, battery measurements, hostile-peer tests and field range remain outstanding.

The user reported the current app working well. That is useful user feedback and must be represented as such; it is not interchangeable with a reproducible two-radio test report. The documentation should invite the next physical trial rather than inflate the evidence.

20. Repository map and build workflow

Location	Main responsibility
`main.go`, `backend/`	Existing hosted server, storage, APIs, authorization, signing and observability
`web/src/apps/`	Office editor implementations
`web/src/lib/offline/`	Local API routing, snapshots, backup/restore and downloads
`web/src/lib/nearby/`	Native bridge, invitation handling, local replica journal and sync protocol
`web/src/lib/crdt/`	Existing structured shared-document and other CRDT foundations
`internal/nearby/`	Native radio wrappers, hub, framing, TLS and tagged relay connector
`cmd/localfirst-offline/`	Stable offline asset server and cache-update startup page
`cmd/localfirst-relay-experiment/`	Separate experimental receiver entrypoint
`experiments/android-relay/`	Native Android relay project and core tests
`web/e2e-offline/`	Offline, recovery, invitation and replica browser scenarios
`scripts/`	Frontend/package/deployment build tooling

Typical source workflow, from the repository's `web` directory on this Windows workspace:

```
npm.cmd ci --no-audit --no-fund
npm.cmd run typecheck
npm.cmd run build:offline
npm.cmd run test:e2e:offline
npm.cmd run package:offline
```

Use the configured portable Go binary or an appropriate installed Go toolchain to run native tests from the repository root:

```
go test ./internal/nearby ./cmd/localfirst-offline
go test -tags relay_experiment ./internal/nearby ./cmd/localfirst-offline ./cmd/localfirst-relay-experiment
```

The Android project needs a compatible JDK, SDK and Gradle setup. At the historical `d7e9135` commit, the inherited unanchored `diwan` ignore rule matched the Java package directory `org/diwan`, omitting local Java source and core tests from that push. Current LocalFirst binary exclusions are anchored as `/localfirst`, and the Java package name `org.diwan` is retained for compatibility. This fixes the current ignore-pattern interaction without rewriting the historical snapshot; versioning the required source and checking an Android build from a fresh clone remain separate release checks.

Workspace-level Android/experimental-Windows build and verification wrappers also live outside the nested repository and are not in that push. Downloaded SDKs, caches and APK outputs are appropriately excluded. The stable Windows `npm` offline build/package scripts are tracked and can be used with their documented prerequisites. The existing checkout retains the folder path `W\diwan`; `git -C diwan ...` selects it from the parent workspace. That filesystem path is independent of the LocalFirst product brand.

Current generated artifacts are kept separate from application source. Backups and historical packages were moved to sibling `W-Archives`, and exact redundant ZIPs were removed after hash comparison. This organization protects the working app; it is unrelated to the cryptographic protocol and should not be presented as a network feature.

21. Failure-oriented walkthrough

Event	Intended behavior	Evidence or remaining qualification
Internet disappears	Local installed Docs editing and saved reopening continue	Offline browser/package checks
Radio is missing/off	Local editing remains usable; bridge/radio error is visible	Native status and UI paths; device matrix remains open
Wrong invitation file	Identify backup/report/invalid input and keep Connect disabled	Parser and browser tests
Previous valid code followed by bad import	Clear the previous code rather than silently reuse it	Regression coverage
Certificate pin or document secret is wrong	Do not expose document offer or accepted updates	Go protocol tests
A relay token is wrong	Do not create a document endpoint connection through that admission	Tagged synthetic relay tests
Relay rewrites/replays ciphertext	TLS authentication/integrity rejects invalid records or terminates the path	Synthetic stream attacks; not a blanket security proof
Destination journal commit fails	No destination durability acknowledgement; show error	Replica/sync fault-injection tests
Two ordinary tabs save	Stale revision rejected; recover local work	Local transaction and browser scenarios

Event	Intended behavior	Evidence or remaining qualification
Two shared tabs compact	Merge committed records before pruning in one transaction	Focused journal tests
Browser update arrives during editing	Startup can activate a current shell without forcibly reloading old editors	Upgrade regression
Browser data is cleared	Local documents may be lost unless exported elsewhere	Explicit product limitation
Android process is killed	Relay ends; endpoints keep their durable local work	Required physical lifecycle validation, not yet passed

22. Terms presenters should be able to explain

Local-first: the user's device has a usable local document and does not require a remote round trip for every edit. **Offline-capable:** an installed build can perform specified work without internet; it does not imply radio communication across arbitrary distance. **Peer:** another participating endpoint. **Relay:** a forwarding device that is not necessarily authorized to read the document.

Snapshot: a saved representation of a document at a point in time. **Journal:** a sequence of durable updates used to recover and merge state. **Checkpoint:** a compacted representation plus a safe boundary for pruning covered journal records. **Revision:** an optimistic-concurrency version for a snapshot save. **State vector:** compact Yjs knowledge used to calculate missing updates; it is not a document hash or a complete deletion acknowledgement.

TLS: the endpoint channel protocol that authenticates and encrypts a reliable stream. **Certificate pin:** the expected hash of the actual host certificate, carried through a trusted invitation. **Bearer capability:** possession grants the corresponding access; it does not identify a human. **RFCOMM:** a Classic Bluetooth stream service, distinct from BLE GATT and Bluetooth SIG Mesh. **SDP:** service discovery that locates a Bluetooth service/channel. **Origin:** scheme, host and port, which determine browser storage and same-origin access boundaries.

23. Practical next engineering milestones

First, complete one physical Windows-host → Android-relay → Windows-receiver trial with both directions edited and destination journal acknowledgements observed. Record device models, Android versions, adapters, drivers, exact artifact hashes, topology, conditions and failure behavior. Repeat using the other phone, then both phones in a chain. Test stop/restart, new tokens, wrong peers, permission revocation and local editing during interruption.

Next, measure rather than assume performance: per-link reliability, usable spacing, end-to-end p50/p95/p99 latency, throughput, memory and battery consumption. Extend route management only after fixed-route correctness and resource limits are understood. A production release additionally needs signed distribution, dependency review, security assessment, reproducible recovery/rollback, accessibility and device matrices, and clear support limits.

No amount of simulated hop count turns into an 800 m field result. No single successful demo proves every failure mode. The strongest technical presentation explains what is built, shows what was actually measured and names the next falsifiable test.

24. Source and evidence ledger

All code links below are pinned to the documented repository edition. Local reports are generated artifacts in the workspace parent and may not be present in a clean GitHub clone.

Topic	Primary implementation or evidence
Dependency declarations and actual frontend resolutions	go.mod, package.json, package-lock.json
Offline build and cache behavior	build-offline.mjs, offline executable, startup script
Snapshot and restore logic	workspace.ts, localApi.ts
Draft and save races	draftStore.ts, filesStore.ts
Structured CRDT validation	ydoc.ts
Durable local replica and acknowledgements	replica.ts, documentSync.ts
Native transport and local HTTP security	bluetooth_windows.go, hub.go, http.go
Pinned TLS and invitation format	secure.go, invitation.ts
Android permissions, forwarding and resource bounds	Committed AndroidManifest.xml; workspace-only RelayCore.java and RelayService.java, absent from the documented GitHub commit because of the ignore-rule gap
Experimental native connector	relay_experiment.go, tagged tests
Browser scenarios and physical checklist	offline scenarios, Windows Bluetooth guide, experiment guide
Recorded package and upgrade results	Local `Diwan-offline-package-verification.json`, `Diwan-offline-upgrade-verification.json`
Recorded relay-lab and prototype results	Local `Diwan-relay-phase1-verification.json`, `Diwan-relay-experiment-verification.json`

Use the source and evidence ledger when a judge asks “Where is that implemented?” A precise reference and a bounded claim are stronger than a broad guarantee.

PART 2 · USP AND EXISTING SOLUTIONS

BIOS2026 / LocalFirst: USP and competitive landscape

Prepared for judges · 1 November 2026 · Evidence baseline: `d7e9135`

This chapter explains what problem this version addresses, which alternatives already solve parts of it, and which differences can be defended today. Competitor capabilities below come from the linked primary sources checked for this guide. They are a targeted comparison, not a feature-complete product audit or a patent novelty search. Capabilities, editions and deployment requirements can change.

1. The problem we are choosing to solve

A team may need to produce a shared report when Internet connectivity is unavailable, unreliable or deliberately excluded. Simply being able to open a document offline does not solve the whole workflow. Users also need to know whether the latest edit reached durable local storage, whether another device received and saved it, and what happens if the connection disappears halfway through an update.

The project explores a local document workflow with three separate responsibilities:

1. **Keep working locally:** the editor, document store and recovery mechanisms should function without an account server or cloud document API in the offline build.
2. **Exchange changes over an explicit nearby link:** the current Windows implementation uses paired Classic Bluetooth RFCOMM and an authenticated TLS session.
3. **Report evidence of delivery honestly:** a connection indicator is different from a destination journal acknowledgement.

The longer-term relay work asks whether participating Android phones can extend that path while forwarding opaque endpoint-encrypted bytes. That is a research and engineering direction; an 800 m or 1 km coverage claim is not a current result.

2. The claim ladder: working, experimental and proposed

Level	Defensible statement	Evidence and boundary
Current local workspace	The Windows offline package can create, edit, save, reopen and export Docs documents using local browser storage.	Packaged-app checks and 19 Chrome scenarios are recorded; this does not certify every editor or all devices.
Current nearby implement ation	The code implements Windows RFCOMM, explicit pairing and invitations, pinned TLS 1.3, opt-in Yjs journals and destination commit acknowledgements.	Native owner listening and invitation download/import were checked on one radio. A complete physical two-laptop document exchange has not been independently recorded.
Simulated protocol evidence	The relay lab exercises end-to-end communication over multiple injected byte-stream hops and rejects several credential, mutation and replay failures.	23 leaf scenarios repeated ten times means 230 test executions, not 230 distinct attacks or physical deployments.

Level	Defensible statement	Evidence and boundary
Experimental Android relay	A separate Java relay APK and a tagged Windows receiver have been built. The phone forwards a manually selected circuit and does not terminate the document TLS session.	Android lint has no issues; JVM and Go protocol checks pass. Actual Android radio, service lifecycle and cross-device relaying remain unverified.
Future network capability	The project targets a measured 800 m route, with a possible 1 km stretch goal, using cooperating nodes.	No measured distance, automatic discovery, dynamic route repair or full-radius coverage result exists.
Production claim	The product has a staged path toward deployment readiness.	It is not production-certified, independently security-audited or supported across a published hardware matrix.

The evidence sources are the local package verification, relay lab report, prototype report and physical test guide. Keep their scopes separate when presenting results.

The difference between feature presence and validation

A feature may exist in source without having passed its physical acceptance gate. A successful software simulation is valuable because it isolates protocol behavior. It cannot prove antenna range, Windows-driver interoperability, Android background survival or two simultaneous radio links on the available phones. This project benefits from making that distinction visible rather than turning every green test into a product claim.

3. A USP that can withstand follow-up questions

The short version

> We extend an open-source office suite into an offline document workflow that separates local saving from peer delivery, adds authenticated Windows Bluetooth sharing, and experiments with consent-based Android relays that forward the endpoints' encrypted stream.

The technical version

The distinguishing engineering focus is the combination of local transactional persistence, recovery-first editing, structured CRDT synchronization, a native non-IP nearby transport, and explicit destination durability acknowledgements. The experimental relay keeps endpoint trust separate from relay admission: a `DRL1` code admits a path, while the `DBT1` document invitation authenticates the actual document endpoint.

The statement to avoid

Do not pitch this as the first offline editor, the first encrypted collaborative editor, the first Bluetooth mesh app, or the only system that works without a cloud provider. Existing products demonstrate all those individual ideas. The project must earn its differentiation through this specific workflow, implementation quality and measured user benefit.

The proposed user benefit

The hypothesis is that a small local group can continue producing a shared document with fewer infrastructure dependencies and clearer recovery/delivery feedback. Whether that is easier, faster or more reliable than alternatives must be tested with users and physical devices. It is a value proposition to validate, not a measured outcome already achieved.

4. What is inherited, and what this version contributes

BIOS2026 is derived from VulOS LocalFirst. The local baseline is commit `8cdb6f5`; the pushed project history preserves that ancestry. LocalFirst already supplied the React office shell, editor integrations, Go server, storage infrastructure, signing-related functionality and existing WebRTC collaboration code. React, Yjs, ProseMirror and TLS are established technologies, not algorithms invented by this project.

The extension work visible against that baseline includes:

- An offline-only build and local API routing for browser-backed documents.
- Transaction-completion-based save reporting, revision conflict checks and token-aware draft recovery.
- Opt-in persistent Yjs replica journals and reconnect synchronization for shared Docs documents.
- A Windows native RFCOMM helper, narrow loopback control API and certificate-pinned document invitations.
- Invitation-file validation and a host document-selection flow.
- An offline-cache upgrade launcher that resolves the old-shell problem without clearing IndexedDB.
- A separate fixed-route relay experiment with Android consent, foreground service controls and bounded forwarding.
- Fault-injection, browser, protocol and package checks that document what has and has not been verified.

These contributions are system integration, persistence semantics, transport adaptation, usability and testing work. The team should demonstrate them using the baseline comparison, rather than claiming authorship of the whole office suite. The inherited MIT and Apache 2.0 license texts and third-party notices remain relevant. This is engineering attribution, not legal advice about every possible distribution arrangement.

Reproducibility gap: the Android Java application and core tests are present and buildable in the current local workspace, but the `d7e9135` GitHub snapshot does not contain all of those Java files because of an existing ignore-pattern interaction. Several workspace build/verification helpers also live outside the main repository. The published repository is therefore not yet a complete, self-contained reproduction of the local Android experiment. Do not call it a fully reproducible public Android release until this is corrected and verified from a fresh clone.

5. Capability comparison

How to read this table: “conditional” means setup, edition, transport or deployment matters. “Not established” means the cited materials did not demonstrate that exact capability; it does not prove the vendor has no such feature. “Not the cited workflow” describes the scope of this comparison. A server on a local network may work without public Internet while still being required for collaboration.

Product or family	Local work without public Internet	Collaboration route in the cited material	Confidentiality relevant to comparison	Fit and gap for this project
BIOS2026 / LocalFirst extension	Implemented and tested for local Docs workflow after package installation	Native Windows Bluetooth; Android fixed-route relay is experimental	Pinned endpoint TLS for document transport; local documents/backups are not application-encrypted at rest	Focused workflow and explicit commit acknowledgements; physical peer/range evidence still incomplete
Google Docs	Yes, after supported-browser/profile offline setup	Service-backed synchronization; offline edits wait for reconnection	Do not infer endpoint-only cryptographic confidentiality from offline availability	Strong familiar collaboration; cited flow does not establish direct Bluetooth co-editing
Microsoft Word / Microsoft 365	Yes, with suitable desktop/mobile app and local document availability	Co-authoring through shared storage such as OneDrive or SharePoint	Depends on product and organizational deployment; not assessed here as a universal E2EE model	Mature desktop editing; nearby serverless document transport is a different target
LibreOffice	Yes, desktop files	Calc has simultaneous shared-file collaboration; other workflows vary	Local ownership does not itself prove encrypted exchange	Strong offline alternative; Calc sharing must not be erased from the comparison
ONLYOFFICE	Yes, Desktop Editors	Connected platform for real-time co-editing	Private Rooms provide a specific encrypted collaboration workflow	Offline editing and encrypted collaboration already overlap with the pitch; Bluetooth path is not established by these sources
Nextcloud Office / Collabora	Desktop/local-file options exist; server-backed online editing requires its service connection	Self-hosted document-server integration, potentially on a private LAN	Self-hosting, transport TLS and file E2EE are different properties	Strong control/administration story; operating without a document server is the narrower difference
CryptPad	Full disconnected editing was not established by the cited sources	Encrypted collaborative documents through an instance	Browser-side end-to-end content encryption is a core feature	A serious privacy-focused office competitor; encryption alone is not a unique claim
Syncthing	Local files remain usable	Peer file synchronization when devices can exchange data	File synchronization security is separate from structured editor semantics	Can preserve conflicting file copies; not the same as operation-level rich-text merging
Briar	Yes, for its supported communication workflow	Direct synchronization over Bluetooth/Wi-Fi; Tor when Internet is available	End-to-end encrypted communication is part of its design	Strong evidence that private offline nearby communication already exists; core focus is messaging

Product or family	Local work without public Internet	Collaboration route in the cited material	Confidentiality relevant to comparison	Fit and gap for this project
Bridgefy / bitchat	Their documented local communication modes do not require Internet	Participating-device nearby/mesh messaging	Review the exact product, message type, version and security documentation	Prior art for device relaying; not evidence that an office-document relay is automatically safe or ready

This is a comparison of workflows, not a ranking of entire organizations. A mature alternative may be the better choice for many real users, especially when enterprise support, complex file fidelity, broad device support or established deployment experience matters.

6. What to say about each alternative

6.1 Google Docs

Google explicitly documents creating, viewing and editing Docs, Sheets and Slides offline after preparation in a supported browser and profile. Therefore “Google Docs stops working when the Internet disconnects” is an incorrect pitch. The narrower comparison is where edits are exchanged: this project's offline build is intended to use a native nearby link rather than rely on restoring connectivity to the document service. That is an architectural distinction, not proof that our editor is more capable or more usable. Google offline instructions

Judge answer: “Google already has offline editing. Our focus is an explicit nearby document-exchange path and local/peer durability feedback. We still need physical validation and user studies to establish the practical advantage.”

6.2 Microsoft Word

Microsoft documents continued offline work and later synchronization, while co-authoring depends on suitable applications, formats and shared storage. Its documented Word workflow can surface conflicts after disconnected edits. Do not reduce Microsoft to “cloud only”: desktop Word is a substantial offline product, and SharePoint deployments introduce additional choices. Our research question is how to maintain a small group's document workflow without relying on a reachable shared document-storage service for the nearby exchange. Microsoft co-authoring requirements and offline behavior

Judge answer: “We are not replacing Word's mature feature set. We are exploring a different transport and persistence workflow for a narrow disconnected-team scenario.”

6.3 LibreOffice

LibreOffice is a direct counterexample to the claim that offline office work is new. Its Calc collaboration documentation also describes shared-file access by multiple users. Consequently, “LibreOffice cannot collaborate” is too broad. The project's intended difference concerns locally persisted rich-text CRDT updates exchanged through the specific authenticated Bluetooth path. That distinction must be demonstrated rather than inferred from the fact that one application runs in a browser and another is a desktop suite. LibreOffice collaboration documentation, shared-spreadsheet setup

Judge answer: “LibreOffice is a strong baseline. A fair evaluation should compare task completion and setup friction against a realistic LibreOffice workflow, not against an artificially restricted version.”

6.4 ONLYOFFICE

ONLYOFFICE documents offline desktop editing and platform-connected collaboration. It also documents Private Rooms, with encryption and particular desktop/server requirements. Therefore neither “office editing without cloud” nor “encrypted collaborative editing” is exclusive to this project. Our comparison should name the exact workflows: Bluetooth endpoint transport and experimental opaque relay circuits here, versus the cited connected-platform and Private Room workflows there. Edition, compatibility and deployment checks belong in an actual selection exercise. Desktop Editors, Private Room requirements

Judge answer: “ONLYOFFICE already overlaps with several privacy and offline goals. Our proposed advantage is the nearby transport workflow, whose benefit still needs measurements.”

6.5 Nextcloud and Collabora

Self-hosted collaboration can keep infrastructure under an organization's control; “no public Internet” does not imply “no server.” Collabora describes a server-based online editing model and separate local/desktop options. Nextcloud documents multiple office integrations, so the engine and release must be specified instead of assuming one universal architecture. Collabora FAQ, Nextcloud office administration

Nextcloud's documented file-E2EE browser limitations should not be confused with ordinary server-side encryption or with every office integration. These are different trust and editing modes. Nextcloud file-E2EE documentation

Judge answer: “A local document server may be a good solution when it is available. Our offline mode aims to avoid needing that shared server for the nearby document path.”

6.6 CryptPad

CryptPad is an especially important comparison because it combines collaborative office applications with browser-side end-to-end encryption and self-hosting. It directly refutes “we are the only private collaborative office suite.” The sources checked here establish those properties but do not establish a fully disconnected Bluetooth document workflow; the table therefore leaves that specific comparison unverified. CryptPad also has its own documented import/export behavior, which should be compared with real test documents rather than assumptions. CryptPad overview, application interoperability FAQ

Judge answer: “CryptPad is closer to our privacy story than many mainstream editors. We differentiate the explicit nearby transport and recovery workflow, while acknowledging its mature encrypted collaboration approach.”

6.7 Syncthing

Syncthing synchronizes files. Its documented handling of simultaneous differing file edits includes conflict copies. This is useful preservation behavior, not a bug to ridicule. A rich-text CRDT addresses a different layer: it represents editing operations inside a document so replicas can reconcile structured changes. That does not make the CRDT automatically superior for arbitrary files, large binary assets or backup use cases. Syncthing synchronization and conflict behavior

Judge answer: “Syncthing is a strong file-sync tool. Our feature is editor-aware reconciliation and destination journal acknowledgement within a document session. We should compare it on that task, not pretend file sync and collaborative editing are identical.”

6.8 Briar

Briar already demonstrates private peer synchronization without a central messaging server, using nearby transports when Internet connectivity is unavailable. Its own roadmap discusses broader distributed applications. It is therefore relevant prior art rather than an unrelated product. The distinction here is the current focus on a structured office document and its persistence/acknowledgement path. It is not a claim that our threat model, metadata protection or operational maturity exceeds Briar's. Briar design overview

Judge answer: “Briar validates the importance of private offline communication. We are adapting that general problem space to an office-document workflow, and our implementation still has important validation gates.”

6.9 Bridgefy and bitchat

Both are relevant examples of nearby or mesh-style messaging, so “phones forwarding messages” is not a new concept. A radio-distance figure advertised for another product must not be reused as our measurement. Likewise, different message types can have different privacy properties. bitchat's project security material explicitly discusses visible broadcasts and proximity observability; that is a reminder to examine the exact traffic model rather than equate “mesh” with confidentiality. Bridgefy FAQ, Bridgefy application explanation, bitchat source, bitchat security overview

Judge answer: “These products are prior art for nearby relaying. Our experiment carries the existing document TLS stream through selected relays, with no current claim of automatic mesh coverage.”

7. Where a defensible advantage could develop

The strongest path is to prove a workflow improvement, not accumulate technology names. A credible pilot might ask a pair of users to edit a small report, disconnect, make independent changes, reconnect and verify both local recovery and destination persistence. The comparison should include the best practical alternative available to those users.

Candidate advantage	What would substantiate it	What would weaken it
Lower infrastructure dependency	Successful setup and document exchange with public Internet, Wi-Fi and cloud services unavailable	Hidden cloud calls, difficult installation, unreliable pairing or an undeclared server dependency
Clearer save/delivery understanding	Users correctly distinguish local commit, connection and peer journal acknowledgement	Users interpret a relay byte counter as proof of document delivery
Safe disconnected editing	Equivalent document state after concurrent edits and reconnection, with recovery paths exercised	Lost edits, silent stale overwrites, blank hydration writes or false acknowledgements
Reduced relay trust	Captured synthetic payloads remain opaque; endpoint identity checks reject a malicious route	Invitation/key leakage, endpoint authentication skipped or readable document fields in route setup
Practical portability	Fresh-clone builds, signed test artifacts and repeatable supported-device results	Missing source, machine-specific build helpers or unsupported radio combinations

These are proposed evaluation criteria. They are not claims of completed user research or statistically established superiority.

8. Target users and use cases: hypotheses to validate

Small classroom or workshop groups: a teacher or student team may need to maintain a shared report during a connectivity outage. A pilot must check whether installing the package and managing invitations is acceptable compared with using an existing offline editor and exchanging files.

Field-reporting teams: a small group may need nearby document exchange where network service is weak. Battery life, physical spacing, device availability and file-size constraints become central. The existing software evidence is insufficient to recommend this prototype for safety-critical operations.

Organizations exploring data locality: a team may prefer local control over documents. However, local storage does not itself provide encryption at rest, remote administration, identity management, retention policy or compliance. Those are product requirements to discover, not capabilities to imply.

A disconnected collaboration research demonstrator: this is the clearest immediate use. The implementation makes transport, durability and trust boundaries tangible, and the tests expose failure conditions that are often hidden by a simple “connected” badge.

For each potential audience, interview users before selecting the final market. Ask what they currently do during outages, how often the problem occurs, how long setup can take, which file formats matter, who owns the devices and who is responsible for backups. Do not present invented adoption counts, market size or willingness-to-pay figures.

9. Practical tradeoffs judges should hear

The current direct-sharing preview has a one-document/one-peer boundary, short-lived invitations and bounded update sizes. A chain of relays requires each adjacent radio link to work. Additional relays can add latency, failure points and battery cost; they do not create bandwidth or repair an empty physical gap.

The Android experiment uses a foreground service with explicit opt-in and Stop control, but no wake lock or battery-optimization exemption. Process termination and power management can break a route. Those choices make participation visible and avoid implying that other people's devices are available as infrastructure without consent.

The proposed 800 m target must become a measured route result under named conditions. An entire 800 m radius would be a different coverage test. Bluetooth range depends on radio/PHY choices, antennas, transmit power, receiver sensitivity and obstacles; another device's maximum range is not our supported range. Bluetooth SIG range explanation

Security also has layers. TLS protects the endpoint channel under its authentication assumptions; it does not eliminate traffic analysis, compromised endpoints, stolen invitations or malicious packet dropping. The TLS 1.3 specification explicitly distinguishes content confidentiality from hiding traffic length. These limits should shape the requirements rather than be hidden in small print.

10. Business and deployment model without invented numbers

The repository's license choices permit an open-source development model subject to their terms and the separate licenses of dependencies. Possible revenue hypotheses include installation support, managed maintenance, organization-specific integration, training and supported hardware deployments. These are options for customer discovery, not current revenue streams or guaranteed licensing conclusions.

A realistic cost model must count engineering support, signing and release infrastructure, device testing, regression maintenance, security review and incident response. “No cloud server for the offline document path” does not mean zero operating cost. Relays consume battery and radio capacity; users incur installation and coordination costs even when no subscription is charged.

Do not price the product during a judge answer unless the number is explicitly identified as an unvalidated proposal. A stronger answer is: “We first need to establish that the workflow solves a recurring problem and that the support cost is manageable. We would then test a support-led offering with a small pilot.”

Production distribution needs complete publishable sources, dependable builds, a secure update path and a support policy. The Android APK is currently debug-signed, so it should be presented as a development artifact. Signing a package identifies its signer and detects modifications; it does not prove that the application is secure.

11. Metrics that would make the next pitch stronger

Question	Measure	Do not substitute
Can users start successfully?	First-attempt task completion, pairing/setup time and reasons for failure	A developer's rehearsed demo
Are edits durable?	Recovery outcomes at each commit boundary, including quota failure and process termination	The fact that a save button was clicked
Are peers synchronized?	Equivalent structured state and destination journal acknowledgements after concurrent edits/reconnect	Connected socket or sent-byte counts
Does the relay help?	Direct versus one-relay versus two-relay success, latency and energy under the same conditions	Number of simulated hops
How far does it work?	Measured node positions and sustained document exchange over a documented route	An 800 m target or a competitor's radio specification
Is it usable under failure?	Recovery success, understandable errors and retained local work	A happy-path screenshot
Is the implementation ready to distribute?	Fresh-clone build, verified signatures, supported-device matrix and resolved security findings	Lint or compilation alone

Keep local typing latency, local commit time, wire propagation and peer durable acknowledgement time as separate measurements. Report distributions such as p50/p95/p99 after choosing a reproducible workload. No such end-to-end performance distribution has been established by the current reports.

12. Hard judge challenges and defensible responses

“What exactly is new if Yjs, TLS and Bluetooth already exist?”

The contribution is their integration into a recovery-aware document workflow with explicit persistence semantics and a native nearby transport. We are not claiming invention of the primitives. The right test is whether this implementation solves the target task more effectively and safely than a practical alternative.

“Is this just a fork with a different name?”

It is a fork, and attribution is part of the presentation. The team should show the inherited baseline and the actual added modules, failure fixes and validation. Claiming the inherited editor or cryptography as original work would be wrong; dismissing all integration and reliability work would also miss the engineering being evaluated.

“Why not use an existing offline office editor?”

That may be the right choice for many users. Our narrower target is concurrent document state and nearby exchange with an explicit peer-commit signal. We need a user comparison to demonstrate that the extra setup is justified.

“Why not use CryptPad or ONLYOFFICE encryption?”

Those are serious overlaps. Our argument must concern the transport and persistence workflow, not the mere existence of encryption. We should also compare their access control, interoperability and deployment experience fairly.

“Why not a local Wi-Fi server?”

A local server or Wi-Fi network is a valid alternative and may offer better throughput. The current offline build intentionally explores a different link that does not require that shared infrastructure. A future transport choice should be driven by measured requirements, not loyalty to Bluetooth.

“Can you use every phone around us as a repeater?”

No. Relays must run compatible software, be powered, have permission and explicitly opt in. This is a cooperative network, not the ability to commandeer discoverable Bluetooth devices. The fixed-route prototype requires selected paired neighbors.

“Have you proved 800 m?”

No. The target is documented and the physical test plan exists. Synthetic multi-hop results isolate protocol behavior but have no distance unit. A supported range requires field measurements with the named hardware and conditions.

“Is every byte secret?”

The document-channel design keeps payload confidentiality at the endpoints, but traffic timing, volume and adjacent radio presence remain observable. Endpoint compromise and possession of an invitation also matter. We do not promise zero metadata leakage or an unbreakable system.

“What stops a relay from lying about delivery?”

A relay-ready byte and relay counters do not satisfy document delivery. The actual destination must authenticate, apply the document update and commit its journal before sending the relevant acknowledgement. Physical malicious-relay validation is still a release gate.

“What can you demonstrate today without asking us to trust a roadmap?”

Show local editing, committed saves, offline reload, recovery behavior, source/test evidence, invitation generation and the separately labeled experimental artifacts. Demonstrate a real two-device or relayed transfer only if it has actually been rehearsed successfully on those devices. Otherwise state the missing hardware evidence explicitly.

“Does passing hundreds of tests make it production-ready?”

No. Repetition, scenario breadth, physical validation and independent review are different forms of evidence. The current reports identify their transport stubs and open gates. Production readiness requires operational and adversarial evidence beyond the present software checks.

“Can someone else build exactly what you showed?”

The stable source is pushed, but the local Android experiment has a publication gap and some tooling is workspace-specific. We should fix those gaps and verify a clean build before promising a complete reproducible Android release. Being explicit about that is stronger than offering a repository link that cannot reproduce the demo.

13. A practical positioning checklist

Before the presentation, make sure every slide can answer four questions: Which mode does this describe? What evidence supports it? What trust or setup assumption is required? What remains unverified?

Keep the headline focused on the document workflow. Keep inherited work attributed. Keep the long-range target in the roadmap. Keep competitor comparisons specific to edition and deployment. Avoid unsupported security adjectives and unmeasured performance numbers. If a judge challenges a claim, show the code path or evidence boundary rather than adding a larger claim.

14. Primary-source reference ledger

All competitor and standards pages below were checked for this documentation on **1 November 2026**. Vendor descriptions establish advertised/documented capabilities; they are not independent benchmarks or security endorsements.

Source	Why it matters
Google Docs offline help	Verifies that offline editing already exists and has setup conditions.
Microsoft co-authoring	Separates offline editing from shared-storage co-authoring.
LibreOffice collaboration	Prevents the false claim that LibreOffice has no collaboration.
LibreOffice shared spreadsheets	Documents shared-location requirements for the Calc example.
ONLYOFFICE Desktop Editors	Offline desktop and connected collaboration overlap.
ONLYOFFICE Private Rooms	Encrypted collaboration already exists in a specified product workflow.
Nextcloud office administration	Office integrations are deployment-specific.
Nextcloud file E2EE	Distinguishes browser/file encryption limitations from ordinary office editing.
Collabora FAQ	Server-based online architecture and local-product distinctions.
CryptPad overview	Browser-encrypted collaboration and self-hosting are prior art.
CryptPad FAQ	Import/export behavior needs product-specific evaluation.
Syncthing synchronization	File conflicts are different from structured CRDT merging.
Briar architecture	Existing encrypted communication over nearby and Internet transports.
Bridgefy FAQ	Nearby mesh communication is existing product territory.
Bridgefy application explanation	A vendor's range example is not this project's field measurement.
bitchat repository	Open-source Bluetooth mesh messaging is relevant prior art.

Source	Why it matters
bitchat security overview	Message category and proximity metadata require careful distinctions.
Bluetooth SIG range	Range is hardware- and environment-dependent.
IETF TLS 1.3, RFC 9846	Endpoint encryption does not hide every traffic characteristic.
Upstream Diwan and project comparison	Separates inherited software from this version's contributions.

PART 3 · PRESENTATION AND DEMONSTRATION

BIOS2026: presentation, demonstration and technical defence

Prepared on 1 November 2026 for the current local development workspace. This is a rehearsal guide, not a record that the proposed physical tests have passed. Read the technical handbook, competitive analysis, and 200-question bank alongside it.

1. The message to remember

Working proposition: an extension of the original open-source Diwan office suite that lets a Windows user keep editing and saving documents locally, then explicitly share a Docs replica over authenticated Bluetooth. A separate Android relay prototype forwards the endpoints' encrypted stream over a manually selected path. Longer range, automatic routing and production certification are future validation work.

Lead with the user's problem: a document should remain usable when Internet service disappears. Then explain the engineering contribution: local commits, recoverable drafts, conflict handling, durable collaborative updates and clear acknowledgement states. Discuss the radio and relay work after the audience understands what is being protected.

The project is not a new radio technology, a new cipher, a new CRDT, or a claim that offline editing has never existed. Its defensible contribution is the integration and validation of a particular workflow. Existing solutions deserve credit for the capabilities they already provide.

2. Three presentation lengths

A. Thirty-second introduction

“BIOS2026 extends the original open-source Diwan office suite for a local-first workflow. A Windows user can create, edit and save a document without an Internet account or cloud document API. For nearby collaboration, we added a native Bluetooth path with authenticated encrypted sessions and durable Yjs updates. We also built an experimental Android forwarding app. The app and protocol have software-test evidence; long-distance radio performance is not yet measured.”

B. Ninety-second pitch

“Our target situation is a classroom, workshop or field team where an Internet connection is unreliable but people still need to work on a document. Traditional offline editors already solve individual editing. Our focus is joining local document ownership with nearby collaboration and recoverable state.

“We started from LocalFirst rather than claiming to build a complete office engine from scratch. The extension adds a separate offline Windows package, browser-local storage, exact save acknowledgements, recovery drafts and a native Classic Bluetooth connection. Shared documents keep a durable Yjs update journal. Reconnecting peers exchange state vectors and missing updates, and the destination acknowledges an update after its journal commit.

“The next prototype uses Android phones as consenting relays. A phone receives a relay access code, not the document invitation, and forwards the original endpoint TLS stream. That is a fixed route, not automatic mesh discovery. We have passing software checks and a built APK; we still need the physical multi-device and range trials. Our next milestone is measured evidence, not an unsupported one-kilometre claim.”

C. Three-minute structure

Time	Show or say	Purpose
0:00–0:30	Describe an Internet outage during document work; introduce BIOS2026 and its LocalFirst base.	Establish the problem and attribution.
0:30–1:15	Disconnect Internet, type in a test document, wait for local save, then reload.	Demonstrate the strongest independently checkable local behavior.
1:15–1:55	Show the offline architecture: browser storage on the same machine; Go helper on loopback; Bluetooth between machines.	Explain why a localhost page can work without Internet.
1:55–2:25	Show the host invitation controls and explain pairing plus pinned TLS. If a physical peer has not passed rehearsal, stop at host setup.	Separate implementation from verified transfer.
2:25–2:45	Show the Android relay build and software-test evidence with an “experimental” label.	Present the extension without treating a simulation as hardware proof.
2:45–3:00	State the next gate: a real Windows–phone–Windows trial, then field measurements.	End with a concrete, testable development milestone.

3. Ten-minute demonstration, minute by minute

Prepare two small disposable Docs documents. Use synthetic text such as “Disaster drill notes — demonstration only.” Do not use personal documents, production credentials or a real invitation in a projected code listing. Keep one person responsible for the keyboard and one for narration if a team is presenting; this is a suggested arrangement, not a claim about actual team membership.

Minute	Demonstration action	Narration and pass condition
0–1	Open the current Windows package and identify the offline workspace bar.	“This is the offline build, served locally at port 8765. The full inherited online product is a separate mode.” The correct UI and document list must appear.
1–2	Disable Internet connectivity and create a Docs document.	“Bundled application assets and IndexedDB let us work without fetching a cloud API.” The editor must become editable after loading and recovery checks.
2–3	Type a paragraph, save, then reload that document.	Explain the difference between typing in memory and a completed local transaction. The same content must reappear with a valid save state.
3–4	Export the workspace and briefly show the restore preview. Cancel the preview unless restoring a disposable copy is part of rehearsal.	“A backup is a separate local file. Restore creates new copies, preserving the existing documents.” Explain that a workspace backup is not a Bluetooth invitation.

Minute	Demonstration action	Narration and pass condition
4–5	Pair laptops beforehand; on the host open Bluetooth → Share this document → Save invitation file.	“The invitation contains the host certificate fingerprint and an authentication secret. It is a capability, so we transfer it privately.” Host listening and invitation download are the minimum verified setup result.
5–6	**Only if the physical test passed rehearsal:** on the receiver select the paired host, load the invitation, connect and open the shared document.	A connected radio is insufficient. Identify the shared document and inspect the endpoint acknowledgement state. If the physical test has not passed, show a clearly labelled protocol/browser test instead.
6–7	**Only on a rehearsed physical link:** edit different text on both endpoints; disconnect the link, edit locally, then reconnect.	Look for equivalent content and authenticated destination commit acknowledgements. Explain that a CRDT preserves mergeable operations; it does not guarantee that the resulting prose is semantically ideal.
7–8	Display the relay diagram and, if available, the Android app's consent, neighbor selection and Stop controls.	“The phone is a relay prototype, not an Android document editor. It does not receive DBT1.” If no device trial exists, use the actual APK/build evidence rather than an animation presented as a live network.
8–9	Show the software evidence summary and its separate hardware column.	Cite 19 recorded Chrome scenarios, the focused test report, the 230 phase-one relay executions, 108 JVM core checks and 27 tagged connector executions in their correct scopes. Do not add them into one misleading coverage percentage.
9–10	Explain the strongest competitor overlap, current limitations and next release gate.	“Offline editing, E2EE and Bluetooth messaging already exist. Our integration targets this document workflow. We still need physical range, battery, device and independent security validation.” Invite the judges' hardest architecture question.

The minute ranges are a rehearsal schedule, not latency promises. A slow or disconnected radio may need more time than the allotted demonstration segment. Do not change a failure into a success claim to stay on schedule.

4. Preparation checklist

Twenty minutes before presenting

1. Launch the exact packaged build you rehearsed. The historical parent-workspace launcher is `Start-Diwan-Offline.cmd`; newly generated LocalFirst packages use their own `Start-LocalFirst-Offline.cmd`. A fresh clone can instead build the Windows package using the commands in section 7.
2. Confirm `http://127.0.0.1:8765/healthz` reports the expected offline server and version. Health only proves the local server is responding; it does not prove radio delivery.
3. Use the same browser profile as rehearsal. Browser storage is scoped to that profile and origin. Avoid private/incognito mode for the working demo.
4. Save and export any valuable work before testing failure scenarios. Use synthetic demonstration documents for all destructive or recovery examples.
5. Check the radios, power supplies and explicit Windows pairing. Keep phones charged if they are part of the trial. Do not assume a phone's model name proves simultaneous-link support.
6. Install only the experimental APK you inspected. It is debug-signed and intended for development trials, not an audited production distribution.

7. Transfer document invitations privately. A projected invitation or copied relay code can grant access within its applicable session and admission conditions.
8. Prepare an offline copy of this guide and the actual evidence JSON reports. Explain recordings as recordings and tests as tests.

Two minutes before presenting

Verify the correct browser tab, display scaling, readable editor, named test document and a clean demonstration route. Keep the stable app on port 8765. The experimental receiver uses port 8768 with a separate browser store; the verification smoke server uses port 8769 and is closed afterward. Opening the experimental workspace does not import the stable workspace's browser documents automatically.

Do not clear browser data to solve a cached-interface problem. Use the updated launcher's `/api/offline-launch` page. That route was chosen because the released old service worker bypasses `/api/`, allowing the native helper to deliver the current launcher even when the cached Home page is old.

5. Failure handling in front of judges

Failure	Immediate response	Explanation that remains accurate
The page shows the old “Nearby sharing not connected” label.	Open the updated launcher page, preserving the profile.	The asset cache is older than the local server; this is different from a failed radio.
Bluetooth is unavailable.	Continue with local editing and report the unavailable radio state.	Offline editing does not require Bluetooth. Nearby collaboration does.
No paired device appears.	Check OS pairing and refresh the device list.	The app enumerates paired devices; it does not silently take over arbitrary devices.
Loading the selected JSON fails.	Check whether it is a workspace backup, connection report or invitation.	Each format serves a different purpose; rejected files must not leave a stale invitation active.
The link is connected but the peer has no current content.	Keep both document editors open and inspect endpoint sync state.	Socket connectivity is not proof of replica synchronization or destination commit.
The local save fails.	Keep the document open and download its recovery copy.	A failed storage commit must not be labelled “Saved.”
A previous-session draft appears.	Restore or explicitly discard it before editing.	Recovery precedes editing so a blank loading state cannot replace unsaved work.
The Android relay stops after screen-off.	Report the observed device behavior and restart explicitly if appropriate.	Power-management and process lifecycle behavior remain physical-device gates.
The relay code expires or a phone restarts.	Generate a fresh code and reconnect the experimental receiver.	DRL1 is session admission; restarting a relay invalidates the old token.
A judge asks for 800 m immediately.	Show the planned measurement protocol and the empty range result.	The current evidence has `measuredRangeMetres: null`; no field result has been established.

An honest bounded demo is stronger than changing the claim after a failure. The best response is: “Here is the state we expected, here is the state we observed, and here is the log or test that will tell us why.”

6. How to answer a difficult question

Use four short parts:

1. **Claim:** state the property you mean, with its mode and scope.
2. **Mechanism:** name the data flow or algorithm that creates that property.
3. **Evidence:** point to a specific source file, test or measured artifact.
4. **Boundary:** explain what that evidence does not establish.

Example: “The nearby protocol acknowledges a received update after the destination journal commit. `NearbyDocumentSync.receive()` applies the update and awaits `LocalReplica.flush()` before sending the acknowledgement. Protocol tests cover failed commits and lost acknowledgements. That does not certify all disk controllers or power-loss cases, and it is not a benchmark of real Bluetooth range.”

If you do not know an answer, name the uncertainty and the next experiment. Do not say that Go, TLS or Yjs automatically makes the entire application correct. Each library has an integration boundary that the project still owns.

Eight questions to rehearse first

Question	Short answer to practise
“What did you build rather than inherit?”	The offline packaging and storage path, recovery/save behavior, native Bluetooth bridge, nearby replica persistence/sync integration, invitation/update usability fixes and the experimental relay work. The editor engines and substantial office/backend code are inherited from the original Diwan project and its dependencies.
“Why not use Google Docs or LibreOffice?”	They are useful solutions with existing offline capabilities. The chosen design targets an explicit browser-local document workflow and a nearby Bluetooth transport that does not rely on a cloud document API. The comparison is about fit, not universal superiority.
“Is your encryption unbreakable?”	No absolute guarantee is justified. The design uses existing TLS 1.3 with a pinned host certificate and invitation authentication, while endpoint compromise, stolen invitations, traffic metadata and denial of service remain concerns.
“Have you achieved one kilometre?”	No. The target is a field-validation goal; simulated relay counts do not measure metres or area coverage.
“Can any nearby device extend it?”	No. A node must run compatible relay software, have power, permission and owner consent, and maintain working adjacent links.
“Is this Bluetooth Mesh?”	The current experiment is an application-level fixed-route RFCOMM overlay. It does not implement the Bluetooth SIG Mesh protocol.
“Does your Android app edit documents?”	No. The current Android experiment is a stream relay. The two Windows endpoints run the document session.
“Can a judge rebuild all of this from GitHub today?”	The Windows offline source/build path is versioned. At commit `d7e9135`, Android Java sources were accidentally excluded by a broad ignore rule and some experiment helpers remain outside the nested repository. That reproducibility gap must be corrected before claiming a clean-clone Android build.

7. Operation and reproduction commands

These commands assume the documented prerequisites are installed. Installing dependencies and downloading SDKs may need Internet initially; running the packaged offline app afterward is a separate claim.

Windows offline source build

From the repository's `web` directory in PowerShell:

```
npm.cmd ci --no-audit --no-fund
npm.cmd run typecheck
npm.cmd run build:offline
npm.cmd run package:offline
```

Node/npm and an appropriate Go toolchain are needed for the build. `scripts/package-offline.mjs` accepts `GO_BINARY`, uses the workspace portable Go if available, and otherwise uses `go`. The resulting complete package includes its launcher; extract/copy the full package instead of copying only its executable.

Read local health without exposing an invitation

```
Invoke-RestMethod http://127.0.0.1:8765/healthz
```

Targeted checks used in this workspace

```
# From the main repository, with Go on PATH:
go test ./internal/nearby ./cmd/localfirst-offline
go test -tags relay_experiment ./internal/nearby ./cmd/localfirst-offline ./cmd/localfirst-relay-experiment

# From web, after building the offline assets:
npm.cmd run test:e2e:offline
```

The parent workspace also contains `Verify-Relay-Phase1.mjs`, `Verify-Relay-Experiment.mjs`, `Verify-Offline-Package.mjs` and `Verify-Offline-Upgrade.mjs`. Those scripts produced the recorded local evidence, but they are not part of the published nested repository at `d7e9135`. Their existence in the workspace is not a clean-clone guarantee. The handbook's reproducibility section records the missing packaging/source work.

8. Evidence to carry into the room

Evidence	What it supports	What it does not support
`Diwan-offline-package-verification.json`	Recorded 19 Chrome cases; packaged executable; local radio owner start/stop; native invitation download/import; offline reload.	A completed two-physical-laptop data exchange.
`Diwan-offline-upgrade-verification.json`	Reproduced old cached interface; current launcher activated new UI while preserving a saved document.	Every possible browser/version upgrade.
`Diwan-relay-phase1-verification.json`	23 distinct simulated relay scenarios repeated ten times, plus scope-labelled baseline checks.	230 distinct scenarios or any measured radio distance.

Evidence	What it supports	What it does not support
`Diwan-relay-experiment-verification.json`	108 JVM checks, nine tagged Go leaf scenarios repeated three times, lint zero, APK signature/permission checks and isolated Windows UI smoke.	Physical Android forwarding, Android document editing, range or production certification.
The historical focused-test summary in the safety audit	A recorded focused run of 133 tests across 13 files.	A fresh full-project test run every time the count is quoted.
Source commit `d7e9135` and upstream base `8cdb6f5`	Reviewable inheritance and implementation delta.	A percentage of original authorship or proof that every locally built artifact is already in Git.

Keep the primary reports locally. This guide summarizes their scope; it is not a replacement for the test outputs or an independent audit.

9. Next milestones and measurable acceptance

The first physical milestone uses one Windows host, one Android relay and one Windows receiver, with the phone maintaining both secure adjacent sockets. Check formatted content and actual endpoint acknowledgements in both directions. Then stop a link, edit locally on both endpoints, reconnect and inspect convergence. Repeat independently on the Samsung A36 and OnePlus Nord, recording exact Android builds rather than assuming support from product names.

Only after that should a two-phone chain be tried. Longer-range trials require surveyed node positions and a working link between every adjacent pair. For a route of length `D` and a measured usable spacing `r`, a simple lower-bound estimate is `ceil(D/r)` links and one fewer intermediate node. That estimate omits obstacles, placement constraints, interference, redundancy and mobility. A route reaching a point 800 m away is also different from full coverage of an 800 m radius.

Performance acceptance needs named hardware and workload: endpoint commit latency, acknowledgement delay, bytes per update, reconnection duration, throughput per hop, battery drain, memory growth and failure recovery. No p95/p99 performance number should appear in the pitch until the measurements exist. Security release work includes independent review, dependency/toolchain checks, source reproducibility, release signing and a tested rollback path.

10. Presentation conduct and ownership

Credit the upstream Diwan office suite and the libraries used for editing, CRDTs and cryptography. Explain that AI assisted implementation and documentation in this development session if asked about tooling. AI assistance does not transfer responsibility for understanding the code, validating claims, handling secrets or reporting failures.

Do not invent user interviews, revenue, deployment counts, benchmark results, certifications or team contributions. For business questions, frame the proposed customer segment and pricing model as hypotheses to test. For production questions, explain which gate has passed and which is still open.

The strongest closing sentence is a specific commitment: **“The next result we need is a repeatable physical Windows–Android–Windows document exchange with destination commit acknowledgements, followed by measured field range.”** It describes a testable milestone without turning a roadmap into a completed feature.

PART 4 · 200 JUDGE QUESTIONS AND MODEL ANSWERS

BIOS2026 / LocalFirst: 200 Judge Questions and Model Answers

Prepared: 1 November 2026. **Source reference:** `d7e9135` in BIOS2026. **Audience:** students, product judges, software engineers, security reviewers and networking specialists.

This is a broad rehearsal bank, not a claim that every conceivable question can be predicted. Answers describe the available source and recorded evidence. Use “implemented,” “tested in software,” “tested on this radio,” and “physically validated between devices” carefully: they are different statements.

The current deliverable extends the open-source original Diwan project. Its strongest demonstrated path is a Windows offline Docs workspace with local persistence and recovery. Native Windows Bluetooth sharing exists as a preview. A separate Android relay APK and Windows receiver have passed software checks. Independent physical two-laptop document exchange, Android relay lifecycle, 800 m–1 km range and production security remain open gates.

1. Beginner questions: problem, purpose and scope

Q001. What is this project in one sentence?

It is a local document workspace that keeps editing and saving available without an internet connection, with a Windows Bluetooth collaboration preview and an experimental Android relay path. The practical idea is that the user's own device remains the primary working copy. Long-distance relay coverage is a development target, not a result we have already measured.

Q002. What problem are you trying to solve?

People can lose a useful collaboration workflow when internet access is unreliable, restricted or unavailable. This project separates local document durability from a network connection, then explores nearby exchange without using an internet server for document transport. It also addresses less visible failures: stale tabs, misleading save indicators, confusing invitation files and interrupted application updates.

Q003. Who is the intended user?

Potential users include students, small teams and field groups who need to continue writing during connectivity gaps and retain control of local copies. Those are target segments, not proven paying customers or deployment partners. Before choosing a market, we would test whether document collaboration is actually their priority and whether they can maintain enough participating relay devices.

Q004. What can I use today?

The Windows package can open its bundled interface, create and edit local documents, save them into browser storage, reopen them and export or restore a workspace backup. It also exposes the native Bluetooth sharing flow. The broader office suite has more inherited features, but our strongest offline verification concerns Docs, so we would not present every editor as equally certified.

Q005. What does “offline” mean here?

For the Windows offline build, application assets are bundled and document operations use local browser storage rather than a remote document API. The browser still talks to a helper on the same computer for Bluetooth controls. Offline does not mean that communication needs no hardware, power or installation; it means the implemented local workflow does not depend on internet service.

Q006. Is this a website or a desktop application?

The interface is a React web application served locally by a Windows executable. The browser provides the editing environment and IndexedDB storage, while the executable serves bundled assets and supplies native Bluetooth access. This combines a reusable web interface with a platform-specific helper. It is different from downloading an unknown public website for the first time with the internet disconnected.

Q007. Is it a replacement for the whole Microsoft Office suite?

That is not the current claim. The upstream codebase includes several office editors, but this development effort concentrates on offline document safety and nearby collaboration. Mature commercial office products have extensive compatibility, accessibility, administration and deployment work behind them. We should compete on a specific validated workflow before claiming an equivalent full-suite replacement.

Q008. What is the main benefit beyond just saving a file locally?

Local saving is only one part. The project adds explicit recovery behavior, revision conflict protection, a durable shared-document journal, visible peer acknowledgement and a nearby connection workflow. The intended benefit is continuing useful work through outages without confusing “a radio connected” with “the other device safely saved my edits.”

Q009. Does the application need a user account?

The offline workspace does not need a cloud account or account server to create and save a local document. The inherited hosted application has separate authentication and deployment capabilities. Account-free local use does not provide account-based access isolation: another person using the same unlocked browser profile can access that profile's local documents.

Q010. Have you achieved an 800-metre range?

No measured 800 m result exists in the available evidence. The relay lab exercised multiple forwarding stages in memory, which proves a limited protocol property rather than a physical distance. Actual coverage depends on adjacent radio links, obstacles, interference, device placement and cooperating nodes. We would only publish a distance after a documented field trial.

Q011. Does the system automatically use every nearby phone?

No. A phone must run compatible relay software, have Bluetooth enabled, grant the required permissions and explicitly opt in. A headset, a discoverable device or a switched-off phone is not automatically a relay. The present Android prototype uses selected paired neighbors and a fixed route; automatic discovery and route selection are later work.

Q012. What should judges remember most?

Our central engineering principle is to make durability and connectivity explicit. Work remains local when the network disappears, and the UI should only claim peer acknowledgement after the receiving journal commits. The Bluetooth and Android relay work explores how to extend that model while keeping document authentication between the actual endpoints, with remaining hardware limitations stated openly.

2. User experience and demonstration

Q013. How does a beginner create and save a document?

The user opens the Windows offline launcher, creates a Docs document and starts typing. The editor reports its local save state, and Ctrl+S provides an explicit save action. A successful state follows local persistence, not an optimistic network animation. The user should keep the same browser profile and origin when reopening the application.

Q014. Where exactly are my documents stored?

The offline documents live in IndexedDB inside the current browser profile for the local application origin. They are not stored inside the ZIP or automatically beside the executable. The stable launcher uses `http://127.0.0.1:8765`. A different profile, private window or port has a different storage context and may therefore show a different workspace.

Q015. Which computer generates the document invitation?

The computer hosting the selected document generates it. The owner opens the document, chooses Bluetooth, starts sharing and saves the invitation file or copies the code. The receiving computer loads that invitation while selecting its paired host. The invitation authorizes access to that session; it is not an export of the document itself.

Q016. How does the host find sharing from Home?

The workspace's Bluetooth dialog includes a document selector and an “Open document to share” action. That opens the chosen document with its sharing dialog available. Hosting still goes through the editor's preparation step so current work is saved before it is offered. The invitation export remains disabled until an active host invitation exists.

Q017. Why are workspace backups and invitation files different?

A workspace backup contains saved document records and history for restoration. A Bluetooth invitation contains session authentication material, not the workspace. Treating the two as interchangeable caused a real usability failure, so the importer now identifies the wrong format and explains how to obtain the invitation. The filenames alone are not trusted; the contents and format version are checked.

Q018. What is the difference between “saved” and “acknowledged”?

“Saved on this device” concerns a successful local persistence path. “Other laptop acknowledged this version” concerns the peer's authenticated document protocol and durable journal commit. A connection state or a relay's byte counter cannot establish the second property. The distinction matters when one device has full storage or disconnects immediately after receiving bytes.

Q019. What happens if the user closes a tab immediately after typing?

The application attempts to preserve a recovery draft and warns about pending work where the browser allows it. Navigation and visibility changes have recovery handling, but browser exit warnings are not a guarantee against process death or power loss. The safe statement is that committed work is durable within the browser's storage guarantees; the newest uncommitted edit can still be lost.

Q020. What does the recovery prompt do?

It asks the user to restore or discard a previously retained draft before editing proceeds. This prevents a newly loaded blank editor from silently replacing recoverable content. A draft is a recovery mechanism, not proof that the normal save succeeded. When storage itself is failing, the interface also offers a downloadable recovery copy where possible.

Q021. How are keyboard users supported?

The implementation includes labelled document controls, keyboard save behavior and modal focus handling, including Escape and focus restoration. Browser scenarios exercise keyboard creation, help and saving. That is useful evidence, but it is not a full accessibility certification. Physical screen-reader, switch input and platform-specific accessibility testing remain part of a wider device matrix.

Q022. How do you handle a user who selects the wrong invitation file?

The parser explains specific errors such as workspace backup, connection report, unsupported version, missing code or oversized file. Importing another file clears the previously entered invitation before reading it, so rejection cannot silently leave a stale credential active. Connect stays disabled until the input passes validation and the required host selection and radio conditions are satisfied.

Q023. Why did an old screen remain visible after an update?

The browser's service worker could keep serving the previous application shell even after the local executable was replaced. The launcher now opens a startup route bypassed by earlier workers, updates the worker and checks its version before opening Home. Existing editor tabs are left open; application code caches and document databases are treated as separate concerns.

Q024. What if the live Bluetooth demonstration fails?

We should show the local offline workflow first, then explain the connection status and failure rather than pretending success. The prepared software tests can demonstrate protocol behavior, but they must be labelled as simulations. For the physical feature, we would record the device and error, keep both local documents intact, and repeat the documented connection checklist after the presentation.

3. Technology stack and engineering choices

Q025. What is the main technology stack?

The interface uses React, TypeScript, Vite and Tailwind CSS, with Zustand for application state. Docs uses Tiptap and ProseMirror, and shared-document synchronization uses Yjs. Go supplies the hosted backend and Windows helper, with native Windows Bluetooth socket calls. The Android relay experiment is a separate Java application built with the Android SDK and Gradle.

Q026. Why use React rather than plain HTML and JavaScript?

The inherited interface already uses React, and the project has many stateful screens: file lists, editors, dialogs, recovery prompts and save indicators. Components make those interactions easier to organize and reuse. React does not provide durability or synchronization by itself, so those responsibilities remain in explicit storage and protocol modules rather than being hidden inside rendering logic.

Q027. Why TypeScript?

TypeScript helps describe file records, save states, native messages and synchronization interfaces before the program runs. For example, a native message has a file identifier and an allowed kind rather than an arbitrary unstructured payload. Static typing improves development feedback, but it cannot make received Bluetooth or JSON data trustworthy; runtime validation is still required at every input boundary.

Q028. Why Go for the helper and backend?

Go fits the existing backend and supports networking, concurrency, standard-library TLS and standalone executable packaging. The helper can serve local assets and own native sockets without requiring the end user to install Node.js or Go. Its goroutines simplify concurrent reads and cancellation, but correct ownership and limits still require careful design and tests; concurrency is not automatically safe.

Q029. Why use a browser interface with a native helper?

The browser supplies the editing stack, storage APIs and a reusable interface, while the helper supplies native RFCOMM access unavailable through the current browser workflow. The boundary avoids rewriting the entire editor for Windows. It also creates a security responsibility: the local helper needs narrow operations, origin checks, authentication tokens and loopback-only exposure.

Q030. Why not implement all Bluetooth access directly in the browser?

The current inter-laptop design uses Classic Bluetooth RFCOMM, while browser Bluetooth support is not a general equivalent to arbitrary native RFCOMM sockets. The existing implementation therefore uses Windows APIs behind a local bridge. A browser-only redesign would need a separately validated transport and capability model; merely changing the UI cannot provide unsupported radio roles or platform APIs.

Q031. What do Tiptap and ProseMirror do?

They provide the structured rich-text editing model and commands behind the Docs interface. A document is a tree of nodes, marks and attributes rather than only a flat string. That structure matters when synchronizing formatting, paragraphs and tables. We reuse those editor primitives and focus our additions on offline persistence, recovery and the nearby synchronization path.

Q032. What does Yjs contribute?

Yjs supplies a CRDT document model, binary updates and state-vector synchronization. It allows independently produced compatible updates to merge without choosing a single authoritative last saved snapshot. The application still has to validate updates, connect the model to the editor, persist its updates and decide when acknowledgement is justified. Yjs does not provide Bluetooth, application authorization or durable storage automatically.

Q033. What is Zustand used for?

Zustand holds application state such as file lists and related operations used by the interface. It is an in-memory state management tool, not the database. The offline API and IndexedDB modules perform persistence, and the editor tracks save generations separately to avoid an old asynchronous save completion marking newer edits as saved. Those layers should not be conflated.

Q034. What is Vite doing in the project?

Vite builds and splits the browser application and provides development tooling. The offline build selects the offline mode, disables the online collaboration path and produces bundled assets plus a generated service worker. The installed user runs those assets locally; Vite is not a required runtime service on the user's laptop. Building and running are separate lifecycle steps.

Q035. Are all listed dependencies part of the demonstrated offline path?

No. The broader repository contains inherited spreadsheet, slide, whiteboard, PDF, import/export and hosted features, so its dependency list is larger than the verified offline Docs path. A good stack explanation identifies what a dependency actually does and in which mode it is used. Package presence alone is not evidence that a feature has been demonstrated or tested.

Q036. How should you report library versions?

Use the manifest to explain declared requirements and the lockfile to identify the concrete resolved packages for a build. A range such as `^18.3.1` is not the same statement as the exact installed React version. For reproducible evaluation, record the source commit, lockfile, toolchain and artifact hashes rather than reading a badge as definitive runtime evidence.

4. Frontend architecture and mode boundaries

Q037. What are the application's main layers?

The interface layer contains React views and editor controls. State and API adapters route file operations to the appropriate mode, while local workspace, draft and replica modules own persistence. The native bridge translates narrow browser requests into local helper operations. Below that, Go manages document sessions and Windows sockets; the Android experiment forwards opaque streams on a selected route.

Q038. How does the offline build avoid calling the cloud document API?

An explicit offline build flag selects the local behavior before the ordinary server request path is used. Supported operations are handled through the local API adapter and IndexedDB, while unsupported operations fail visibly. The offline build also disables online endpoint priming and online Docs collaboration. Browser request monitoring checks that the tested workflow makes no external or cloud document requests.

Q039. Is the inherited WebRTC collaboration path used for Bluetooth?

No. The repository retains its existing online or self-hosted peer collaboration architecture, but the offline build disables that Docs path and uses the separate native nearby interface. Bluetooth document updates travel through the helper's RFCOMM connection. Describing Bluetooth as WebRTC without internet would hide a real architectural difference and confuse both connectivity and trust boundaries.

Q040. What is lazy loading, and why use it here?

Lazy loading delays editor engines and other large modules until their screens are opened. This reduces the JavaScript that must execute just to display Home. The offline build still bundles and precaches the relevant application assets for later disconnected use. A smaller initial entry does not prove that total storage, all editor memory usage or typing latency has become equally small.

Q041. What is a service worker?

A service worker is a browser-managed script that can intercept requests for its origin and serve cached application resources. Here it supports reopening the app without reaching an internet server. It is not the document database and does not make local files immune to storage deletion. Its lifecycle also needs careful handling so an older cached interface does not hide a newly installed build.

Q042. Why not save private documents in the application cache?

Application caches and document storage have different purposes and upgrade lifecycles. The service worker caches code, styles and other application assets; document records belong in IndexedDB with explicit transactions and recovery logic. Mixing documents into an asset cache would complicate access control, updates and deletion. Keeping them separate also makes it easier to explain what an application-cache replacement does to user work.

Q043. What does the error boundary protect against?

It catches certain React rendering failures and presents recovery actions instead of leaving a completely blank application. It does not catch every asynchronous error or repair a corrupt database automatically. Its purpose is to keep the user oriented and provide a route back to files or reload, while the storage layer separately controls the integrity of saved document records.

Q044. How do you stop blank content overwriting a loading document?

The editor remains read-only until file hydration and recovery checks are complete, and save preparation checks that a document is ready. An empty initial editor is not treated as an intentional edit to an existing document. This is a lifecycle problem as much as a storage problem: a perfectly valid database write can still destroy useful work if it writes the wrong loading state.

Q045. How do you prevent an old save result from reporting newer text as saved?

The save flow records a dirty generation before starting persistence. If the user edits while that save is pending, the generation changes and the completion must leave the document dirty. Draft cleanup also checks its exact token. Together, those checks prevent an earlier operation from acknowledging or deleting recovery state belonging to a later edit.

Q046. Why not show “synced” whenever the Bluetooth connection is active?

A live socket says that a transport exists; it says nothing about validation, application of the update or a receiving storage failure. The application therefore distinguishes link state, local save state and peer acknowledgement. This is especially important during reconnects and tests: an attractive green connection indicator would be misleading if the peer's journal had not committed the latest deletion.

Q047. Can multiple tabs affect the same document?

Yes, so the design must treat tabs as concurrent actors rather than assuming one editor owns a record. Ordinary snapshot documents use revision checks to reject stale replacement writes. Shared replicas use a journal and cross-tab notifications, with compaction merging committed records inside the transaction. Tests cover several tab interactions, but that does not establish every possible browser scheduling behavior.

Q048. What happens when an operation is unavailable in offline mode?

It should fail explicitly rather than returning a fake success or silently using an internet fallback. The offline interface hides or disables services that do not belong to that mode. The inherited online application may support more features, but those should not be advertised as available merely because the same codebase contains their components or server handlers.

5. Local persistence, recovery and database behavior

Q049. What local databases are used?

The main workspace database is `diwan-local-workspace`, with stores for files, folders, metadata and versions. Recovery drafts use `diwan-drafts`, while shared document checkpoints and updates use `diwan-replicas`. These databases serve different responsibilities: snapshots and file organization, last-resort draft recovery, and the durable CRDT history needed for shared-document replay and resynchronization.

Q050. What fields are in a local file record?

A record includes its ID, name, type, content, revision, ownership marker, parent folder, starred and trashed flags, and timestamps. Shared documents also carry nearby-related flags. The content field is not a universal plain-text blob: each editor may use its own representation. Validation and migrations must therefore respect the document type and schema rather than assuming every record is interchangeable.

Q051. Why use IndexedDB rather than localStorage for document content?

IndexedDB supports asynchronous operations, structured values, indexes and transactions, making it more suitable for multiple document records and journals. localStorage is synchronous and does not supply the same transaction model. The choice does not remove quotas or eviction risks, so the application must still handle storage denial, write failures, profile changes and the need for separate backups.

Q052. When does a database operation count as successful?

The workspace and replica transaction helpers resolve their result after the transaction completes. Receiving an object-store request's success callback alone is not enough if the enclosing transaction later aborts. This distinction is important for user-visible save state and for sending a peer acknowledgement: both must describe committed work, not merely a write request that was accepted temporarily.

Q053. What does strict transaction durability mean?

For relevant read-write transactions the application requests IndexedDB's strict durability option. It is an expression of the desired storage behavior, not proof against hardware failure, browser defects or every power-loss scenario. We still distinguish an acknowledged transaction from an uncommitted edit and keep power-loss testing as a release gate. Storage guarantees must be explained in the actual browser and platform context.

Q054. How is a stale snapshot write detected?

The update includes the revision that the editor believes it is modifying. Inside a transaction, the storage layer compares it with the current record's revision and rejects a mismatch instead of blindly replacing newer work. The error can include the current record for recovery decisions. This protects against lost updates but is not the same thing as automatically merging every concurrent edit.

Q055. Why distinguish ordinary documents from shared replicas?

Replacing a whole snapshot is reasonable for an ordinary local file when revision checks protect concurrent tabs. A shared document needs mergeable operation history, so it explicitly opts into a durable Yjs replica. Keeping that migration explicit limits the effect on existing local files. An older snapshot editor is rejected after migration rather than being allowed to overwrite the new replica-managed document.

Q056. What happens when a file is missing?

The storage adapter reports that the document was not found, and the editor should not become editable as though an empty replacement were the original. This matters because a blank document is valid data and could otherwise be saved successfully under the wrong assumption. The missing-file test checks the visible failure and verifies that editing or saving a blank replacement is not enabled.

Q057. How are recovery drafts different from versions?

A draft is a recent recovery copy retained around an unfinished or failed save, while versions represent prior committed snapshots in the workspace history. A version does not necessarily contain the most recent unsaved typing, and a draft does not establish that the ordinary document record was committed. Each has separate cleanup, restore and user-interface behavior.

Q058. Why attach a token to a draft?

Consider save A writing a draft, then the user making edit B before A completes. If A deleted the draft by document ID alone, it might erase B's newer recovery copy. The token identifies the specific draft associated with A, so cleanup deletes only that record if it still matches. Explicit user discard uses a separate intentional path.

Q059. What happens when both document storage and draft storage fail?

The interface must remain honest about the failure and preserve the current editor content as far as possible. A recovery download provides a path that does not rely on the same failing IndexedDB write. Fault-injection browser tests exercise that behavior, but the downloaded file is still the user's responsibility to save and retain. Closing the editor prematurely can remove that remaining recovery opportunity.

Q060. What does a workspace backup include?

The backup is a structured representation of saved local workspace data, including files, folders and relevant history and deleted-state records. It is separate from the application executable and the Bluetooth invitation. The import process validates the complete structure before committing restoration. A source-code checkpoint is another different artifact and does not automatically back up personal browser documents.

Q061. Does restoring a backup overwrite current files?

The intended restore flow adds new copies with new identities rather than replacing the user's current workspace. It previews what will be restored and preserves existing work. This makes a mistaken restore more recoverable and avoids treating old backup revisions as authoritative current state. Repeating a restore can produce more copies, so that behavior should be explained instead of silently deduplicated.

Q062. What are the important storage limits?

The workspace code limits individual document content to 16 MiB and retains at most 50 stored versions. Backup restore is bounded at 32 MiB, with limits on record counts and nesting. Nearby sharing has smaller limits of its own. These are application safeguards, not claims that the browser always has that amount of free storage or that every near-limit file has equivalent performance.

Q063. Can browser storage be deleted even after a successful save?

Yes. A user can clear site data, lose the device or use a different browser profile; storage pressure and browser policy also matter. The app can request persistent storage and report the actual result, but it cannot make the data immortal. A separate exported backup remains necessary for work that must survive device or profile loss.

Q064. Does changing from port 8765 to 8768 expose the same documents?

No, ports are part of a web origin, so those addresses use separate browser storage contexts. The experimental receiver deliberately uses port 8768 to isolate its workspace from the known working app on 8765. Users should not interpret an empty experimental file list as evidence that the stable documents were deleted. This separation is a safety boundary, not a synchronization feature.

6. CRDTs, journals and synchronization logic

Q065. What is a CRDT in simple terms?

A conflict-free replicated data type represents changes so compatible replicas can combine independently produced updates according to defined rules. For a collaborative document, this lets two devices edit while disconnected and later exchange missing information. It does not mean every user's intended meaning is automatically preserved; the application must still choose an appropriate document model and show understandable results.

Q066. Why not just send the latest entire document and replace the other copy?

If both devices edit independently, replacing one whole snapshot with the other can discard valid work. Shared replicas instead exchange Yjs updates and merge them. Full-state updates may still be used for synchronization barriers, but they are CRDT state encodings applied through the CRDT machinery, not an instruction to overwrite a peer's unrelated concurrent operations with a plain JSON snapshot.

Q067. Does using a CRDT guarantee that no edit is ever lost?

No. The convergence property assumes that relevant valid updates survive and eventually reach compatible replicas. A device can still lose an uncommitted edit, a database can fail, or an update can be rejected for exceeding limits. The journal, validation, recovery and acknowledgement layers are necessary because a mathematical merge model alone does not provide durable delivery or availability.

Q068. How is the rich-text editor connected to Yjs?

The document uses a structured Yjs context integrated with ProseMirror rather than treating rich text as arbitrary string offsets. Updates can represent structural edits and formatting. Incoming data is validated before touching the live document context. This avoids the assumption that an insertion offset in a serialized representation always corresponds to the same meaningful rich-text location on another replica.

Q069. What is a state vector?

A Yjs state vector summarizes how far a replica has seen the insertion histories associated with client identities. A peer can use it to compute missing update information. It is useful for resynchronization after disconnection, but it is not a full document hash and does not by itself describe every deletion. Our acknowledgement design explicitly accounts for that limitation.

Q070. Why do state vectors need special treatment around deletions?

Two states can have the same insertion-clock summary even though their deletion knowledge differs. If the application used only equal state vectors to declare synchronization, it could falsely acknowledge a deletion that the receiver had not committed. The protocol therefore tracks update IDs and coverage barriers, and deletion-related tests check that old acknowledgements cannot silently stand in for newer changes.

Q071. What is stored in the replica journal?

The replica database stores binary Yjs updates associated with a file and optional checkpoints containing combined state. These records support reconstruction when an editor reopens. The journal is local to an endpoint and does not depend on an intermediary holding the only copy. A relay's responsibility is different: the current prototype streams bytes without persisting document payloads.

Q072. Why serialize journal writes through a promise chain?

Local editor changes and incoming updates can arrive while earlier persistence is pending. The replica's chained write sequence gives an explicit order for completing those operations and propagating failures. Its `flush()` drains the current tail, including work appended while an earlier tail was awaited. Otherwise, a save or acknowledgement could describe only part of the locally applied state.

Q073. How does replay work after restarting an editor?

The replica reads its checkpoint and committed updates, validates and applies them to the Yjs context, and only then permits normal editing to continue. Replay must not be mistaken for a fresh user edit that creates an endless persistence loop. A corrupt or invalid replay is a visible recovery problem; silently starting with an empty document would hide potential data loss.

Q074. What is compaction and why is it needed?

Without compaction, the journal would keep accumulating incremental records. After a bounded number of writes, the code combines state into a checkpoint and prunes the updates covered by it within the same transaction. It includes other committed tab records before pruning, which prevents one tab's incomplete view from erasing operations already stored by another tab.

Q075. Is journal compaction the same as deleting old edits?

It replaces a representation of committed CRDT state with another representation covering the same information needed for replay. It must not discard concurrent committed updates that were absent from the current editor's in-memory view. Product version history is a separate concept; compacting the synchronization journal is not a promise to retain every individual human editing action as a navigable historical version.

Q076. How does the protocol decide what to send after reconnecting?

A new session or resynchronization event starts a new synchronization generation and exchanges durable state vectors. Each endpoint computes update information missing from the peer and sends it through the authenticated document session. Pending transmission bookkeeping can be rebuilt from persistent CRDT state. The design does not require an in-memory outbound queue to survive every disconnection.

Q077. Why use a generation counter in synchronization?

Asynchronous sends, reads and storage completions may finish after a session has disconnected or changed. The generation counter identifies the connection context to which a task belongs. Before publishing results or sending an acknowledgement, the code checks that the generation is still current. This prevents stale callbacks from marking a new session as synchronized or using obsolete connection state.

Q078. What are the native document message kinds?

The browser protocol uses `vector`, `update` and `ack`, with a file identifier and encoded data. Updates include IDs used for acknowledgement tracking. The helper validates the selected document and allowed message kind before transmitting. This separation gives resynchronization summaries, actual CRDT changes and durability acknowledgements distinct meanings instead of overloading a generic “success” packet.

Q079. When does the receiver send an acknowledgement?

After validating and applying the update, it waits for the replica persistence chain to flush. It sends an acknowledgement only if that operation succeeded and the session generation is still current. The meaning is a committed endpoint journal, not that a relay accepted data or that an operating-system socket buffer received it. Later UI acknowledgement checks also consider outstanding and pending local updates.

Q080. What is the `coversEarlier` acknowledgement barrier?

It marks an update whose acknowledged coverage can retire preceding outstanding update records. During resynchronization, a delta derived against the peer's durable state can cover earlier changes; when tracking grows too large, a full-state update is used as a barrier. This is safer than discarding old tracking entries merely because their acknowledgements have not arrived, particularly for deletion-only changes.

Q081. Are repeated updates a problem?

Yjs is designed to merge duplicate compatible update information idempotently. That helps when reconnect or retransmission repeats data, but it does not mean arbitrary replay is an authorization mechanism. TLS protects the stream against record tampering and unauthorized ciphertext replay, while the application still validates document identity, message bounds and acknowledgement state. Duplicate-tolerant CRDT behavior and cryptographic freshness solve different problems.

Q082. What happens when an incoming update is invalid?

It is rejected instead of being blindly applied to the live editor. The Yjs validation path checks supported document structure and relevant bounds, and the synchronization layer reports an error without claiming acknowledgement. Rejecting malformed content may interrupt progress, which is preferable to corrupting the document silently. The user still needs a recovery route for their valid local committed state.

7. Windows Bluetooth and local bridge

Q083. Which Bluetooth transport is actually implemented?

The Windows implementation uses native Classic Bluetooth RFCOMM sockets. It is not a Wi-Fi tunnel renamed Bluetooth, and it is not the Bluetooth SIG Mesh protocol. The browser communicates with its own loopback helper; document traffic between laptops uses the native radio path. The old bounded BLE framing module should not be confused with the current transport selected for the Windows preview.

Q084. What is RFCOMM?

RFCOMM provides a stream-oriented communication model over Classic Bluetooth that applications can use somewhat like a connected byte stream. The current implementation places TLS and application framing above it. Similarity to a TCP programming model does not make the physical transport IP or Wi-Fi; platform-specific discovery, pairing, socket options and radio behavior still apply.

Q085. Why must the devices be paired?

The Windows path enumerates OS-authenticated paired devices and requires an appropriate paired target before dialing. Pairing also supplies the operating-system Bluetooth security layer requested by the sockets. That relationship is not the complete document authorization: the receiver must additionally authenticate the intended document host through its certificate pin and invitation secret.

Q086. What is an SDP service UUID?

A service UUID identifies the Bluetooth service being advertised or requested during service discovery. The document service and experimental relay service use different UUIDs so their protocols are not accidentally mixed. A UUID is an identifier, not a password. A device advertising a matching service must still pass the relevant pairing, admission and endpoint authentication checks.

Q087. Why is the helper bound to loopback?

It is intended to serve the browser and control Bluetooth only for the local machine. Binding to a loopback IP avoids exposing its HTTP bridge as a general LAN service. The implementation also checks the Host header and request origin context. These are separate defenses because a local network address restriction alone does not prevent malicious browser content from attempting requests.

Q088. What prevents any web page from controlling the helper?

The bridge rejects inappropriate origins and fetch-site contexts, validates loopback host usage and requires a generated native token for protected operations. The token is obtained through the same-origin bootstrap path. This reduces cross-origin abuse but does not protect against arbitrary malicious code already running in the trusted application origin or a fully compromised local computer.

Q089. What HTTP operations does the native bridge expose?

Its narrow surface includes bootstrap, status, paired-device enumeration and event polling, plus commands to host, connect, stop and send validated document messages. It does not expose a general command shell or arbitrary filesystem access. Request methods, JSON shape, message size and one-command parsing are checked so the browser bridge is a small control interface rather than an unrestricted native capability.

Q090. How does the browser receive native Bluetooth events?

The native bridge uses cursor-based event requests with bounded waiting rather than requiring the browser to own the Bluetooth socket directly. The helper maintains bounded event history and signals when the cursor no longer covers the retained events. A reset tells the document layer to resynchronize from durable state instead of assuming every intermediate notification is still available.

Q091. Why is there length-prefixed framing above RFCOMM?

A byte stream does not inherently preserve application message boundaries. The native code prefixes each JSON packet with a four-byte big-endian length so the receiver knows how many bytes form one message. It rejects zero, oversized and truncated frames. The two-MiB outer frame bound is separate from the smaller validated document-update and initial-snapshot limits.

Q092. What happens if a socket write sends only part of a frame?

The writer keeps sending the remaining bytes until the frame is complete or an error occurs. A zero-progress write becomes a short-write failure instead of causing an endless loop. On the receiving side, full reads are required for the header and body. These details matter because assuming one write equals one read would break even on an otherwise healthy stream.

Q093. Why are there deadlines and cancellation paths?

An unresponsive or malicious peer can otherwise hold sockets and goroutines indefinitely. The helper uses bounded setup and communication deadlines and owns pending as well as established connections so Stop can interrupt them. Cancellation also prevents old connection attempts from publishing a connected state after a newer user action has replaced the session.

Q094. What limits apply to a Windows sharing session?

The preview supports one selected document and one peer, with a source JSON snapshot up to 512 KiB and a replica/update bound of 1 MiB. The native outer frame has a 2 MiB maximum, and invitations/sessions are limited to 30 minutes. Those limits should be stated as current design constraints, not interpreted as measured throughput or radio range.

Q095. Has the real Bluetooth hardware been tested at all?

Yes, the recorded single-laptop checks verified an available radio, paired-device handling, listener and service registration behavior, and real host start/stop with invitation export/import. That is more than a pure socket mock, but it is still not a completed two-physical-device document exchange. Hardware evidence must be described at the level that the test actually exercised.

Q096. What is the next physical Bluetooth acceptance test?

Use two identified Windows laptops with Wi-Fi and internet disabled, exchange a real invitation, edit in both directions and verify endpoint journal acknowledgements and content convergence. Then test concurrent edits, disconnection, reconnect and restarts. Record device, OS, driver, artifact version and failure conditions. The Android relay trial adds another independent pair of physical links and lifecycle conditions.

8. Encryption, authentication and trust boundaries

Q097. What does end-to-end encryption mean in this design?

The document TLS session is established between the actual document endpoints, while relays forward its encrypted byte stream without terminating that TLS layer. The intended document content is therefore not plaintext at a normal forwarding relay. This statement assumes uncompromised endpoints and correct authentication; it does not claim anonymity, invisible traffic metadata or immunity to every implementation vulnerability.

Q098. Which encryption algorithm do you use?

The native document connection requires TLS 1.3 and uses Go's TLS implementation to negotiate an allowed TLS 1.3 cipher suite. We should not claim a single hardcoded record cipher merely from the certificate type or the word TLS. The session certificate uses an ECDSA P-256 identity, while record encryption and key establishment are separate parts of the protocol.

Q099. Did you invent your own encryption?

No. The implementation relies on standard TLS primitives rather than inventing a new cipher or key exchange. Our engineering work is in how endpoint identity is distributed, how the document invitation is checked, how the stream is relayed and how durable acknowledgements are interpreted. Standard cryptography is necessary but still requires correct integration, implementation review and release-specific vulnerability assessment.

Q100. What is inside a DBT1 invitation?

It encodes a 32-byte SHA-256 pin of the host's session certificate and a random 32-byte invitation secret, represented as a versioned URL-safe code. The certificate pin tells the receiver which endpoint identity to accept; the secret authorizes joining that host session. It is not a plaintext copy of the document or a TLS traffic-key export.

Q101. Why is the session certificate self-signed?

The offline workflow cannot depend on a public certificate authority issuing a certificate for each nearby session. Instead, a trusted out-of-band invitation pins the exact certificate generated by the host. That changes the trust model: the user must get the authentic invitation to the intended receiver. A public CA is replaced by explicit pin distribution, not by an absence of identity checking.

Q102. Why does the code set `InsecureSkipVerify`?

The name refers to bypassing the ordinary CA-chain and hostname verification path for this self-signed offline identity. The code supplies a `VerifyConnection` callback that checks the expected certificate pin in constant time. Removing that callback or accepting any self-signed certificate would be a serious flaw. The defensible explanation includes the replacement verification, rather than pretending the field is not present.

Q103. Does the certificate pin authenticate the human owner?

It authenticates the session certificate whose pin was placed in the invitation. That only becomes meaningful as an owner identity if the invitation was distributed authentically to the intended person. The protocol does not establish a government identity or automatically prove who is holding an unlocked laptop. Human verification and invitation handling remain part of the trust boundary.

Q104. Why require a secret if the certificate is already pinned?

The certificate pin helps the receiver reject a wrong or impersonating host. The separate invitation secret lets the host reject a client that has not been given session access before offering the document. These are opposite directions of trust. The current flow is bearer-invitation authorization, not mutual certificate-based human identity with an enterprise account directory.

Q105. Can someone who steals the invitation join?

Possession of a valid invitation is an important authorization capability, so it must be treated as a secret. A person who also satisfies the relevant Bluetooth access conditions could attempt to join during its valid session. The UI therefore warns about sharing and expiry. Encryption cannot compensate for giving an access credential to an unintended person.

Q106. Do the sender and receiver share a fixed document encryption key?

The DBT1 invitation carries a certificate pin and authentication secret, while TLS negotiates the traffic keys used by the connection. It should not be described as simply copying one fixed document key into the JSON file. Relays receive neither the endpoint invitation nor the negotiated TLS keys. An accurate answer distinguishes authentication credentials from the session's actual encryption key material.

Q107. How is a man-in-the-middle rejected?

A relay or other intermediary presenting its own certificate does not match the pin in the receiver's authentic invitation. The handshake is therefore rejected before the document session is accepted. Synthetic tests exercise that behavior. If an attacker can replace the out-of-band invitation itself, the original identity assumption has already failed, so invitation distribution must also be protected.

Q108. Can a relay alter the document secretly?

Changing protected TLS record bytes should cause authentication failure rather than producing an accepted altered application message. The lab tests ciphertext mutation and rejection, but that is limited implementation evidence rather than a proof of all possible attacks. A relay can still drop, delay or interrupt traffic. Integrity protection does not force a malicious forwarding node to deliver data.

Q109. Does encryption hide the amount of traffic?

Not completely. A relay can observe adjacent devices, connection timing and the volume and sizes of data flowing through it. Even encrypted communication can reveal patterns, such as when users become active. Padding could reduce some inferences, but it introduces bandwidth and battery costs and would need separate design and measurement. We do not claim zero metadata leakage or anonymity.

Q110. Are documents encrypted at rest?

The local workspace and exported JSON backups are not encrypted at rest by this application. Browser profile and device security therefore matter. A locked device or operating-system disk encryption can contribute protection, but that is not equivalent to an application-level encrypted database design. We should not use end-to-end transport encryption as an answer to a local-storage privacy question.

Q111. Can the hosted server read stored document content?

Do not generalize the Bluetooth trust model to the hosted persistence mode. The inherited hosted application has file persistence APIs and storage backends; their saved document content and authentication model must be assessed separately from encrypted peer-transport frames. “The collaboration stream is encrypted” does not prove that every server-side snapshot, import, log or backup is opaque to the operator.

Q112. How are replay attacks handled?

TLS authenticates records in the context of a connection, and recorded ciphertext is not a reusable application authorization token for a different session. The lab exercises duplication and substitution of previous-session ciphertext, while application generation and acknowledgement checks prevent stale asynchronous state from declaring a new session complete. Invitation reuse and future route-control replay need their own lifecycle and expiry policies.

Q113. What security claim can you defend today?

We can point to explicit OS-paired links, pinned TLS endpoint authentication, invitation-secret checks, bounded frame validation, bridge access checks and recorded negative software tests. We can also show an APK that lacks Internet permission and does not request a document invitation. We cannot claim a completed independent audit, perfect secrecy of all metadata, secure compromised endpoints or production certification.

Q114. What would production security review need to cover?

It would examine the threat model, invitation distribution and revocation, native bridge, TLS integration, parsers, Android permissions, lifecycle races, relay admission and resource limits. It would also assess logs, crash dumps, update authenticity, dependencies and device compromise assumptions. Findings need reproducible fixes and regression tests. Passing a short demo or using a standard cipher does not replace that review.

9. Android relays and cross-platform implementation

Q115. What exactly does the Android application do?

The experimental Android app forwards a stream between two explicitly selected paired devices. It is not currently an Android document editor or a complete automatic mesh. The first intended physical route is a Windows receiver through a phone to a Windows document host. A second phone can extend a manually configured chain, subject to physical simultaneous-link and lifecycle validation.

Q116. Why implement the Android relay in Java?

The prototype can use native Bluetooth sockets, Android foreground services and permission handling directly without embedding the office editor or a large additional framework. Java also allows the platform-independent admission and copy core to be tested on a JVM. That choice does not settle the eventual Android endpoint architecture; document editing on Android is still a separate implementation decision.

Q117. What permissions does the APK request?

It requests legacy Bluetooth permissions on the older API range, Bluetooth connect access, notifications and the foreground-service permissions needed for a connected-device service. The artifact check found the expected six permissions and no Internet permission. It does not request location or contact access. Actual grant, revocation and OS-version behavior still require tests on the intended physical phones.

Q118. Why is there a foreground notification?

Relaying is an ongoing activity that the phone's owner should be able to see and stop. The app starts an explicit foreground service with a Stop action rather than silently relying on a hidden background process. A notification is not a guarantee that Android will keep the process alive under every condition, and disabled notification access is handled as a setup problem.

Q119. Can relay mode run when the app screen is not open?

The intended mechanism is the explicitly started native foreground service, so forwarding need not depend on keeping the activity visible. However, this has not been validated across the reported phones' screen-off, battery-saver and manufacturer-specific process management conditions. Force-stop, process death or a disabled radio can end the relay. We should never promise an inactive or unpowered phone will continue forwarding.

Q120. What is the DRL1 code?

DRL1 is a separate admission credential for an experimental relay circuit, based on a random 32-byte token. It permits the selected incoming neighbor to open the phone's forwarding path. It is not the DBT1 document invitation and does not give the relay endpoint TLS keys. Separating those credentials lets the relay enforce access without being authorized to read the document.

Q121. What happens during relay admission?

The incoming stream presents the `DRL1` magic followed by the token. The phone verifies the bounded header, the selected incoming device and bonded state, then establishes its configured outgoing connection. It returns the ready byte after that path is ready. Document TLS authentication still happens beyond the relay, so a ready relay does not imply a valid document session or saved payload.

Q122. Why are relay neighbors selected manually?

A fixed route keeps the first prototype's trust and failure behavior understandable: each phone owner chooses the allowed incoming device and a specific outgoing target. It avoids advertising automatic route discovery before that subsystem exists. Dynamic routing would require authenticated discovery, expiry, loop prevention, fairness and malicious-node analysis, all of which are additional protocol work rather than a small UI enhancement.

Q123. Can the incoming client tell the phone to connect to any target?

The current phone is configured by its owner with the allowed neighbor and outgoing device, rather than accepting arbitrary destination commands from the incoming client. That reduces the risk of turning a relay into a general Bluetooth proxy. It does not make relay access safe under every pairing configuration; resource exhaustion and compromised local software still belong in the threat model.

Q124. What are the prototype's relay limits?

There is one active circuit per phone, with 16 KiB copy buffers in each direction and a shared 64 MiB circuit byte budget. Relay mode expires after 30 minutes, with 12-second setup and 30-second inactivity limits checked by a watchdog. These are safeguards for the experiment, not measured sustained throughput or production capacity figures.

Q125. Why use one shared byte budget for both directions?

If each direction or thread enforced its own unsynchronized count, concurrent copying could exceed the intended resource bound. The relay core uses an atomic shared budget so both directions reserve from the same limit before forwarding a chunk. JVM tests exercise that concurrent accounting. The forwarded-byte counter is updated after a successful output operation and still does not mean a document commit occurred.

Q126. Does the relay decrypt and re-encrypt each document chunk?

It does not terminate the document TLS session. It forwards opaque bytes while the actual document endpoints authenticate and exchange protected records. There can also be OS-level security on each paired radio link, but that is a separate layer. Calling every hop's link encryption “end-to-end encryption” would be incorrect if the relay were given plaintext between those links.

Q127. How does a two-phone route get configured?

Configure the phone nearest the host first, with the host as its outgoing device. Configure the previous phone to use that phone as the next relay and provide its DRL1 code. The Windows receiver uses the first phone's relay code and still receives the host's DBT1 invitation directly. Adjacent devices must be paired and able to sustain the necessary physical links.

Q128. Are Samsung A36 and OnePlus Nord officially supported?

They are the user's reported available test devices, not a certified support matrix. Exact variants, Android versions, firmware and simultaneous Bluetooth behavior need to be recorded during trials. The app displays model and Android/API information to help with that process. Building an APK with a minimum SDK does not prove every device above that API level behaves identically.

Q129. What does a debug-signed APK prove?

It proves that the development artifact has a verifiable signature and can be identified by its recorded hash and application ID. It does not mean the artifact is production-signed, store-reviewed, penetration-tested or physically validated. The report identifies it explicitly as an experimental debug build. A release signing and update strategy remains part of production preparation.

Q130. Why does restarting a relay require a new code?

The prototype creates a fresh admission token for the new relay session and does not silently restore the old one from persistent storage. The incoming Windows experiment must therefore be restarted or reconfigured with the current code. That is less convenient than automatic recovery, but it avoids pretending a stale credential still authorizes the new service instance. Improved credential lifecycle design is later work.

10. Failures, adversarial conditions and edge cases

Q131. What happens when Bluetooth disconnects during editing?

The local document remains editable according to its local storage and recovery state. Shared updates are retained through the endpoint journal rather than relying on a connected relay queue. Reconnection starts resynchronization and only a valid receiving acknowledgement can restore the peer-acknowledged state. The physical behavior across actual range loss still needs the two-device acceptance trial.

Q132. What if a relay deliberately drops every packet?

Encryption cannot force delivery. The circuit eventually fails or times out, while the endpoints retain their local committed work and do not claim successful synchronization. Dynamic alternate-route selection is not implemented in this fixed-route prototype. Detecting malicious routing behavior and choosing another path without creating new security problems is part of the later routing phase.

Q133. What if a relay reorders bytes?

The current endpoint protocol expects a reliable ordered byte stream. A forwarding layer must preserve that property or close the circuit; it cannot arbitrarily shuffle TLS record bytes and expect the document layer to repair them. A future datagram transport would need its own ordering, retransmission and bounds design below the endpoint stream, or a different reviewed protocol above it.

Q134. What if the connection is very slow?

The application should expose pending synchronization rather than claiming immediate delivery. Setup and inactivity deadlines bound resource use, so a sufficiently slow or interrupted path may fail instead of hanging indefinitely. Timeouts are therefore a tradeoff between usability and protection. We need field latency and throughput measurements before selecting production values for long multi-hop paths.

Q135. What if the document becomes too large while editing?

The nearby replica enforces its current bounds and reports a recoverable error rather than sending unbounded data or falsely acknowledging it. The user should retain or download a recovery copy. The ordinary workspace's larger document limit does not make the document eligible for Bluetooth sharing. Product messaging must show which limit applies at the specific operation being attempted.

Q136. What if two devices create unrelated documents with the same ID?

The receiver checks whether an existing local record with that identifier is an appropriate shared replica. A conflicting ordinary, trashed or different-type record is rejected rather than overwritten. Random UUIDs make accidental collisions unlikely, but validation is still necessary because received identifiers are untrusted input. Probability is not a substitute for explicit collision behavior.

Q137. What if an old editor saves after the document is converted to a replica?

The local workspace rejects a non-replica snapshot write to a record that is now managed as a shared replica. This prevents an older tab from replacing journal-based work with a stale whole-document snapshot. The user receives recovery guidance and should reopen the current editor. Migration correctness needs this backward interaction check, not only a successful first conversion.

Q138. What if the peer's storage is full?

Applying an update in memory is not enough to acknowledge it. If the destination journal flush fails, the protocol reports the error and does not send a successful durability acknowledgement. The sender keeps its local committed state for recovery and resynchronization. This is why a packet-receipt counter or a radio-level acknowledgement cannot replace application-level persistence confirmation.

Q139. What if a received JSON message contains extra fields or multiple objects?

The native HTTP command decoder disallows unknown fields and checks that the request contains one complete command rather than trailing JSON. It also enforces content type and a body-size bound. These checks make the bridge's intended request grammar explicit. Other parsers have their own validation boundaries; strict command parsing should not be generalized to every unrelated import format without inspecting it.

Q140. What if the invitation expires while the host is still open?

The native session has a bounded lifetime and reports expiry. The host must start sharing again to create a current invitation, and the receiver must use that new credential. The interface should not offer an inactive invitation as though it still worked. Expiry is tied to the application's session lifecycle, not merely the self-signed certificate's separate validity interval.

Q141. What if an old native event arrives after reconnecting?

The bridge and synchronization code associate events and asynchronous work with session or generation state. Messages for a different selected document or obsolete connection are not supposed to advance the current session. Tests exercise cancellation and stale-state cases. This is necessary because asynchronous delivery can outlive the UI action that originally started it, even when the underlying byte stream is ordered.

Q142. What if a malicious peer sends a huge length header?

The native framing reader checks the length before allocating the full body and rejects values outside its bound. The browser update path also checks encoded and decoded update sizes. Defense in depth matters because a request can pass one layer's grammar while being too costly or invalid for the next. Resource limits are part of security as well as performance.

Q143. What happens if Android revokes Bluetooth permission mid-session?

Native calls can throw a security exception even after an earlier permission check succeeded. The prototype catches permission-related failures, closes owned sockets and reports a stopped or failed link rather than treating the previous check as permanent permission. Lint and code review cover the explicit handling; testing real revocation during forwarding remains a physical Android lifecycle gate.

Q144. Can a stop action leave a hidden socket running?

The design owns listener, pending and established resources so stopping can close them, cancel work and prevent stale connected states. The Android review also corrected ownership of accepted sockets before permission-sensitive inspection. Software checks reduce the risk but are not a proof of all OS race schedules. Repeated stop/start, process-death and physical radio-transition tests remain necessary.

11. Testing and evidence quality

Q145. What tests have actually passed?

The recorded stable package report lists 19 Chrome scenarios and real single-radio host start/stop plus invitation export/import. Earlier focused verification recorded 133 tests across 13 files. The relay lab has 23 scenarios repeated ten times; the prototype verifier adds 108 JVM checks, nine tagged Go scenarios repeated three times, zero Android lint issues and an isolated Windows package smoke test.

Q146. Why not combine all those counts into one impressive test total?

They measure different things and some runs repeat the same scenario. Adding them into one unqualified number would obscure the difference between a distinct test, a repeated execution, a static check and a physical observation. We report each layer separately with its transport assumptions. A hundred simulated hops do not become a hundred verified Bluetooth links by arithmetic.

Q147. What do the Chrome scenarios demonstrate?

They exercise actual browser behavior such as local creation, saved-state recovery, backup restoration, stale tabs, invitation handling and offline reload or restart. Some collaboration scenarios replace only the native transport with a test double so real editor and persistence code can be checked deterministically. That proves browser/protocol behavior under the tested conditions, not physical radio performance between laptops.

Q148. What is fault injection?

It deliberately introduces failures such as unavailable IndexedDB, write errors, missing documents or rejected imports so recovery behavior can be tested. Waiting for those failures to occur naturally would make coverage slow and inconsistent. The limitation is that a synthetic fault approximates a class of failures; it does not prove that every real disk, browser or operating-system failure behaves identically.

Q149. What do the relay captures prove?

The lab captures synthetic streams in memory at each relay and checks that selected document, invitation and payload markers are absent in both directions. This supports the claim that the tested implementation forwards protected endpoint traffic rather than those plaintext markers. It is not a cryptographic proof, an anonymity analysis or evidence that production crash dumps and all logging paths have been audited.

Q150. Have you tested a real Android phone forwarding a document?

No physical Android relay exchange is recorded as passed in the available evidence. The APK compiles, passes lint, has a verified debug signature and permission set, and its platform-independent core has JVM tests. Those are valuable preparation steps, but simultaneous radio links, power management, user permissions and endpoint delivery must still be exercised on the actual phones.

Q151. Why is the Go race-detector gate still open?

The attempted race build failed because the installed local MinGW compiler cannot compile the required Windows amd64 runtime. A normal passing Go test run is not a substitute for a race-detector pass. The report records this toolchain limitation explicitly. A compatible 64-bit compiler and realistic concurrent workloads are needed before that specific check can be marked passed.

Q152. How would you make the tests reproducible for judges?

Provide the source commit, lockfiles, toolchain requirements, test command and expected report, with no private credentials or personal documents. There is a current reproducibility gap: the broad `localfirst` ignore rule accidentally excludes the local Java package directory, and some build/verifier helpers live outside the pushed repository. Before claiming a reproducible Android source release, correct that rule, commit the missing source and make the helpers portable. Label each transport as memory, loopback, browser stub or real Bluetooth, and record the exact hardware for field trials.

Q153. Why test the actual packaged executable?

Unit tests cannot reveal every packaging failure, such as missing assets, a stale frontend version, incorrect worker behavior or an unexpected runtime dependency. The package check runs the executable with a restricted runtime environment, opens a fresh browser profile and checks relevant behavior. It complements source tests; it does not convert a local packaging smoke test into proof of multi-device interoperability.

Q154. How do you test an application update without losing work?

The upgrade verifier starts from an older cached application, creates saved work, switches to the current server and reproduces the stale interface. It then opens the special launcher and checks the new controls, the unchanged old editor and a subsequent offline reload. That tests a concrete earlier failure while preserving the distinction between asset cache activation and personal document storage.

Q155. What should be in a physical field-test report?

Record device models, exact OS and driver versions, artifact hashes, route and adjacent distances, radio settings, obstacles and start/end times. Include exact-content or convergence checks, endpoint commit acknowledgements, latency and throughput measurements, disconnects and recovery outcomes. Avoid publishing document content, invitation secrets or private device identifiers. A map and byte counter alone do not establish durable collaboration.

Q156. How do you decide that a feature is production ready?

Define supported devices, correctness, recovery, security and performance gates before declaring success. Production readiness requires release signing, update and rollback behavior, vulnerability review, operational support and hardware evidence in addition to passing software tests. The current relay experiment has explicit open gates. We can demonstrate progress without relabelling those remaining risks as already solved.

12. Performance, scale and range

Q157. What is the maximum Bluetooth range of the project?

It has not been measured as a supported product limit. Bluetooth range depends on the actual radios, physical layer, power, antennas, obstacles and interference, as described by the Bluetooth SIG. Our current software tests do not provide a metres-per-hop number. The 800 m to 1 km figures are targets for later trials.

Q158. How many relays are needed for 800 metres?

That cannot be answered without measuring reliable adjacent-link spacing in the intended environment. As arithmetic only, 800 m with usable 40 m spacing would require at least 20 links and 19 intermediate nodes on a straight path. That is not a recommendation or a measured laptop range, and redundant coverage, obstacles

and route shape can require more nodes.

Q159. Is an 800-metre route the same as an 800-metre radius?

No. A chain connecting two endpoints along one route establishes neither all directions nor every point within a circle. Radius coverage requires a sufficiently connected distribution of nodes across the area and trials that identify gaps. This distinction changes both hardware cost and deployment planning, so the project should not use a successful linear route to advertise circular area coverage.

Q160. Does adding relays always improve performance?

Relays can make a previously unreachable endpoint reachable, but each hop introduces scheduling, radio contention, failure opportunities and energy use. A longer path may have worse latency and lower useful throughput. The result depends on actual hardware and traffic. That is why hop-count tests are a protocol exercise while performance targets must be established through measured field workloads.

Q161. Can the current release support many simultaneous collaborators?

The Windows Bluetooth preview is explicitly limited to one selected document and one peer, and the Android relay prototype permits one active circuit per phone. The inherited online collaboration stack has different capabilities and constraints. We should not borrow a scalable architecture diagram from another mode as evidence that this particular radio preview already supports a classroom full of concurrent editors.

Q162. What performance metrics would you measure first?

Measure local typing responsiveness and save-completion latency separately from end-to-end update acknowledgement latency. Add connection setup time, reconnect time, throughput, memory growth, CPU use and phone battery drain. Report distributions such as p50, p95 and p99 under a named workload and hardware setup. Averages alone can hide the occasional long stall that makes collaborative editing frustrating.

Q163. Is the 25-millisecond outgoing timer a latency guarantee?

No. It is a coalescing delay before attempting to flush pending persisted updates. Actual completion also includes storage, scheduling, serialization, Bluetooth transport, peer validation, receiving storage and acknowledgement. Those stages can dominate the timer. We should explain configuration constants as design choices and then measure the user-visible latency rather than presenting the smallest timer as a benchmark.

Q164. Why send periodic state-vector resynchronization?

It provides another opportunity to discover missing durable update information after connection changes or missed application-level progress. The code schedules it every eight seconds while the selected document is connected. That interval is not a maximum convergence guarantee and does not compensate for a permanently unavailable route. A healthy path and successful endpoint persistence remain necessary for acknowledged synchronization.

Q165. Does a 16-KiB buffer mean the relay uses only 32 KiB of memory?

No. Those are the explicit copy buffers for two directions, not the whole process footprint. The Android runtime, threads, Bluetooth stack, sockets, UI and other objects also use memory. The byte budget bounds traffic permitted for a circuit rather than total retained memory. Real device profiling is necessary before making an overall memory or low-end-device suitability claim.

Q166. How would you avoid a relay being overwhelmed?

The prototype starts with one circuit, bounded buffers, shared byte limits, admission checks and setup/idle/session timeouts. A more general network also needs global admission limits, fairness, backpressure, routing-message limits and protection against connection floods. Increasing concurrency before those controls and measurements exist could turn an availability feature into a resource-exhaustion problem on participating phones.

Q167. Why not use Wi-Fi for higher bandwidth?

Wi-Fi-based local transport can be a valid alternative and should be evaluated against the actual requirement. The current effort explores Bluetooth-only nearby communication, which has different access and power constraints. We should not pretend Bluetooth is always the best transport. If users prioritize large files, throughput or fixed-site coverage, a different or explicitly selectable transport may be the better design.

Q168. Would dedicated radio hardware solve the range problem?

It could change the available link budget, deployment control and supported physical layers, but it would also change cost, installation and interoperability. We have not validated or recommended a particular hardware product. The current reported test setup uses two laptops and two phones. Any dedicated-radio extension would need its own transport, security and field-test evidence rather than inheriting the existing range target as a guarantee.

13. Build, deployment and operations

Q169. What does an end user need to install on Windows?

For the packaged offline app, the user extracts the complete Windows package and runs its launcher, keeping the bundled web assets beside the executable. Node.js and Go are development requirements rather than runtime requirements for that package. Bluetooth sharing additionally requires an available supported radio and pairing. A clean disconnected-machine installation remains a separate acceptance scenario from running on the development laptop.

Q170. What does the Android user install?

The current local artifact is the separate experimental relay APK, identified by its application ID and hash. It is not the main Windows editor packaged for Android. The user must grant the relevant permissions, select paired neighbors and explicitly start relay mode. The locally built APK is not yet a complete reproducible public source release because Java sources were accidentally ignored; production distribution, signing and update trust also remain unfinished.

Q171. How are the stable and experimental applications isolated?

The known working offline app uses port 8765, while the experimental Windows receiver uses 8768 and a distinct artifact directory and banner. The different origins separate their browser workspaces. Go build tags exclude experimental connector code from the normal build. This isolation lets testing proceed without silently changing the runtime behavior or document storage context of the user's working release.

Q172. How do you identify exactly which build is running?

The local health response reports frontend cache and native executable versions, derived from build artifacts. The launcher compares those values with its package metadata instead of treating any responding server as current. For evaluation, also record the source commit and APK or executable hashes. A version number identifies an artifact; it does not by itself prove the feature's hardware behavior or security quality.

Q173. What role do Vercel and Railway play?

The repository includes a cloud deployment route with a frontend preparation step and a Go server deployment configuration. Those are separate from the offline Windows app's local asset server and Bluetooth helper. A cloud deployment URL is useful for a hosted mode but is not a dependency of the implemented offline document workflow. Deployment presence also does not certify cloud authentication, availability or cost.

Q174. Are cloud credentials stored in the project source?

The deployment configuration is designed to obtain relevant secrets through environment or local configuration rather than hardcoding real credentials into publishable source. The main-repository publish review excluded local runtime data and credentials. That review is not a perpetual guarantee about every future commit. Secret scanning, least privilege and deployment-specific configuration remain necessary operational practices.

Q175. Why were SDK files not committed to Git?

Downloaded toolchains, caches and generated application packages are not the project's source. Trying to stage them caused excessive line-ending warnings and a Windows path-length failure, so local exclusions now keep those directories out of staging. Source exclusion needs equal care: an existing broad `localfirst` ignore rule also matched the Java package name, which is an accidental gap to fix. Reproducible setup should version the actual source and tool definitions while downloading SDKs by documented versions and checksums.

Q176. Why does the repository contain both hosted and offline code?

The project extends an existing self-hostable office suite, so its original backend and online collaboration features remain useful context and functionality. The offline build selects a different persistence and transport path without requiring all inherited code to be deleted. Documentation must make the active mode explicit; otherwise, features from one deployment can easily be misrepresented as guarantees of another.

Q177. How should updates preserve user documents?

Keep the same browser origin and profile, update application assets separately from document databases, and avoid reloading unsaved editors automatically. The launcher and service-worker update path address a specific stale-cache problem. Future schema changes require explicit migration and rollback testing. Replacing an executable or clearing a cache should never be explained to users as permission to clear their personal browser data.

Q178. What operational work remains before release?

Release signing, update authenticity, rollback, clean installation, conflicting ports, supported device matrices and privacy-respecting diagnostics need complete validation. Native and Android dependency/toolchain review also remains important. The current debug and preview artifacts can support controlled experiments, but deployment convenience is not the same as a supported production service with recovery procedures and maintained security updates.

14. USP, alternatives and differentiation

Q179. What is your defensible USP?

The strongest candidate is the combination of an offline document workspace, explicit local durability and recovery, endpoint-authenticated Bluetooth collaboration, and an experimental relay path that does not require relays to receive document invitations. Each component has precedents, so we should present the specific integrated workflow and evidence rather than claim that offline editing, CRDTs or encrypted Bluetooth are inventions unique to this project.

Q180. Why would a user choose this instead of Google Docs?

For a user whose requirement is account-free local ownership and a Bluetooth-only collaboration path, this architecture explores a different workflow from a cloud-first office service. Google Docs also has offline capabilities, so claiming that it cannot work offline would be inaccurate. The honest comparison concerns installation, collaboration transport, trust and validated constraints, not a blanket statement that one product has every advantage.

Q181. How is it different from desktop Word or LibreOffice?

Desktop office tools already provide strong local editing and mature document features. Our additional focus is the nearby authenticated sharing path, durable replicated document state and visible distinction between a local save and peer commit. We should acknowledge their compatibility and usability strengths. A judge may reasonably prefer an existing tool if the nearby collaboration requirement is not important to the target user.

Q182. How is it different from ONLYOFFICE or Collabora?

Those are established collaborative office alternatives with their own deployment models and product-specific security options. Our demonstrated offline path places local storage on the endpoint and uses a native Bluetooth document session rather than assuming a particular hosted document collaboration service. Exact edition and configuration matter when comparing encryption or offline features; we should not reduce a competitor's complete product family to one simplified deployment.

Q183. How is it different from CryptPad?

CryptPad is a relevant privacy-focused collaborative alternative, so encrypted collaboration alone is not a sufficient uniqueness claim. The project's proposed distinction is the bundled local workspace and native nearby/relay workflow under connectivity constraints. The comparison should also recognize that a mature privacy product may have stronger operational and security experience. We need evaluated user benefits, not merely a different implementation language or protocol.

Q184. Is this just another file-transfer application?

A file-transfer workflow can move a completed document snapshot, but it does not automatically merge concurrent rich-text edits or distinguish destination journal commits for a shared CRDT state. Our shared-document protocol aims at that ongoing collaboration behavior. Transfer tools remain useful for distributing the installer, backups or invitations, and may be sufficient when users only need to exchange completed files.

Q185. Why not just use Syncthing?

File synchronization can be very useful, but synchronizing files is a different level of abstraction from a structured collaborative editor. A file-sync tool does not automatically supply this editor's schema validation, CRDT integration, draft recovery and peer-acknowledgement semantics. We should still acknowledge that users with ordinary file workflows may prefer a mature sync tool instead of adopting a new document application.

Q186. What about Bluetooth messaging or mesh applications?

They are relevant alternatives for communication without normal internet service, and they show that offline relaying is not a new category invented here. Our proposed application is durable rich-text document collaboration rather than only exchanging messages. Their routing, security and platform capabilities must be compared from current official documentation and tested configurations. We should not infer feature gaps from their names or marketing descriptions.

Q187. Can you claim this is the first such project?

No systematic prior-art study in this work establishes a worldwide first. The code also extends an existing open-source suite and uses established editor, synchronization and cryptographic libraries. A more defensible statement is that this version integrates and tests a particular offline durability and nearby-sharing workflow. Novelty claims should be narrower than “nobody has ever combined these technologies.”

Q188. What would prove that your difference matters to users?

Run the same realistic outage-and-collaboration task with target users across suitable existing tools and this prototype. Measure completion, lost work, setup errors, recovery success, acknowledgement understanding and participant effort, not just technical packet delivery. For relays, include the burden of installing, pairing and powering intermediate devices. A technically different transport is not automatically a product advantage.

15. Ownership, business and project responsibility

Q189. Did you build the entire office suite from scratch?

No. This repository extends the original open-source Diwan codebase, with a known upstream baseline and retained history. The contribution should be explained through the actual offline workspace, safety fixes, native nearby transport, invitation/update improvements, relay prototype and tests added or changed here. Libraries and upstream authors should be credited explicitly instead of presenting the inherited editor and backend as entirely original work.

Q190. What licensing responsibilities follow from that?

The repository identifies an MIT-or-Apache-2.0 licensing model and contains license and third-party notice material. We need to preserve applicable notices and review the actual licenses of redistributed dependencies and artifacts. Open source permits many uses but does not mean every attribution or distribution obligation disappears. Commercial distribution and branding questions need a release-specific review rather than an unsupported blanket legal claim.

Q191. What is the business model?

A business model has not been validated by the available engineering evidence. Possible hypotheses include supported deployment, training or managed institutional workflows, but each requires customer interviews, cost analysis and willingness-to-pay testing. We should not invent revenue, customers or grants. First demonstrate that the workflow is valuable enough to justify its installation and hardware participation costs.

Q192. Is this cheaper than existing solutions?

No measured total-cost comparison has been completed. Local and open-source operation can change licensing or hosting costs, but support, deployment, device power, updates, backup and maintenance still cost time or money. A dense relay deployment may also require many participating devices. Cost claims should compare a defined use case and include those operational requirements rather than only the absence of a cloud subscription.

Q193. What is the biggest project risk?

For the longer-range concept, the biggest immediate risk is turning a correct stream-relaying prototype into a reliable physical network on ordinary phones and laptops. Simultaneous links, Android lifecycle, placement and user participation can prevent a usable route even when encryption and merge logic work. Security hardening and durable recovery are equally important release risks that a distance-only demonstration would not resolve.

Q194. What would you do next with more time or funding?

Complete a controlled Windows–Android–Windows physical exchange first, followed by a two-phone fixed chain and recovery scenarios on recorded hardware. Then measure energy, latency and range before deciding whether dynamic routing or a different transport is justified. In parallel, close the race-toolchain and independent security-review gaps. Expanding features should follow evidence instead of making the architecture broader before its essential path is reliable.

16. Expert traps and difficult follow-ups

Q195. If the system is end-to-end encrypted, why can a relay still identify adjacent devices?

End-to-end encryption protects the document session's content and integrity, not every fact about its physical transmission. A relay must interact with its neighbors and can observe when and how much data it forwards. Hiding network participation and traffic patterns would require additional privacy mechanisms and a separate threat model. We therefore do not claim that transport encryption makes the network invisible or anonymous.

Q196. If your state vectors match, why is that not sufficient evidence of synchronization?

A state vector summarizes insertion progress, while deletion knowledge can differ without changing that summary in the same way. The application therefore also tracks authenticated update IDs, pending writes and acknowledgement coverage barriers. A judge can inspect the deletion-related tests and `coversEarlier` logic. The right answer is that document-state convergence and durable receipt require more evidence than a convenient summary comparison.

Q197. Can you guarantee availability, consistency and partition tolerance all at once?

We should not answer by treating a slogan as a feature checklist. During a partition, local replicas can keep accepting compatible local edits and later converge when updates survive and communication resumes; they do not provide a single instantaneously consistent global view. Durability and authorization are separate properties. The product must show that state honestly rather than calling disconnected replicas synchronously consistent.

Q198. What happens if the receiving peer lies about committing an update?

The acknowledgement is authenticated as coming from the endpoint session, but transport cryptography does not prove the remote software executed a durable storage operation honestly. Our implementation sends acknowledgements after its journal flush; a malicious modified endpoint can violate that behavior. The trust model therefore includes endpoint software integrity. Trusted hardware attestation or stronger remote-proof schemes would be different, unimplemented requirements.

Q199. Does passing 230 relay executions prove the design is secure and can reach a kilometre?

No. Those executions repeat 23 distinct synthetic relay scenarios under controlled memory-stream conditions. They help detect specific protocol regressions, but they do not measure radio propagation, Android power management, hidden implementation races or all cryptographic threats. The report explicitly leaves physical range and production/security certification false. A strong presentation values that scope precision more than an inflated test count.

Q200. Why should judges value the project if so many production gates remain open?

Because useful engineering progress can be evaluated before a production launch, provided its boundaries are clear. The project demonstrates concrete offline storage and recovery behavior, a native authenticated sharing implementation, an isolated relay prototype and reproducible software evidence. Its next steps are specific and falsifiable. The credible claim is a working foundation with a disciplined path to validation, rather than a promise that every difficult problem is already solved.

Evidence and source map

These links identify the main implementation boundaries behind the answers. Some older repository documents describe an earlier milestone; where they disagree with current source or a newer verification report, use the narrower current evidence.

Topic	Primary project source
Scope, upstream attribution and repository history	Project README, upstream Diwan, source reference `d7e9135`
Offline build and service-worker behavior	Offline build generator, offline mode flag, native launcher
Local file schema, revision validation and backup limits	Workspace storage, local API adapter
Draft tokens and save generations	Draft store, file state/store
Journal, checkpoint replay and atomic compaction	Local replica
Update IDs, generations, vectors and durability acknowledgements	Document synchronization, synchronization tests

Topic	Primary project source
Native bridge and access checks	Browser native bridge, Go HTTP bridge
TLS identity, invitation secret and native packet framing	TLS/framing implementation, native session hub
Windows socket transport	RFCOMM implementation, Windows guide
Invitation file validation	Invitation parser, parser tests
Android permissions and relay-only scope	Android manifest, experiment guide
Android admission, byte budgets and lifecycle	Relay core, relay service
Synthetic relay and endpoint trust tests	Relay lab, tagged relay tests
Open gates and measured-versus-planned distinction	Relay phase plan, scenario register

Public-source completeness note: the Android Java links above point to files present in the local working tree. At the documented commit `d7e9135`, the root ignore pattern `localfirst` also matched the Java package directory, so the Java implementation and core tests were not included in the pushed source. Several Windows/Android build and verification helpers remain outside the nested repository too. The locally built APK and software reports are real evidence, but a clean-clone Android rebuild is not yet a passed claim. Correct the ignore scope, commit the missing implementation and make build helpers portable before presenting the Android source release as reproducible.

The workspace-local evidence files are `Diwan-offline-package-verification.json`, `Diwan-offline-upgrade-verification.json`, `Diwan-relay-phase1-verification.json` and `Diwan-relay-experiment-verification.json` in the parent workspace. They are development reports, not independent certificates, and are not automatically part of the public source repository. The package report records 19 Chrome scenarios; earlier audit prose mentioning 18 is historical. The prototype report records 108 JVM checks, nine tagged Go leaf scenarios repeated three times, zero lint issues, debug-signature verification and isolated Windows smoke checks. Its physical-relay, Android-lifecycle, range and production fields remain unverified.

For external comparison facts, use the primary sources and dated comparison notes in the companion project documentation. Do not memorize unsupported competitor absolutes such as “Google Docs has no offline mode,” “all other products expose plaintext,” or “Bluetooth mesh is our invention.”